

Building a Minimal Command-Line Internet Client in C

A Step-by-Step Tutorial

S.F.

July 2026

Abstract

This tutorial guides you through the creation of a compact yet powerful command-line tool (`urlget`) capable of fetching web pages over HTTP and HTTPS, with optional Tor support via SOCKS5. Development is incremental: each step adds a focused piece of functionality, explains *why* it exists, and shows the complete source so far—with prior code muted in gray and new code highlighted—so you always know what changed and where it lives.

Foreword

High-level libraries are wonderful until you need to know what they are actually doing. This tutorial takes the opposite path: we talk to the network with ordinary C, POSIX sockets, and a thin slice of OpenSSL. The point is not machismo. It is demystification.

Almost bare-metal networking is not hard. Creating a socket, connecting it, writing bytes, and reading bytes is a short, repeatable recipe. Once those four moves are clear, you already understand the bulk of what every network client does under the hood. Making the same path meaningfully faster in user space usually means dropping to assembly (or accepting that the kernel, the NIC, and physics own the rest of the budget). Clarity first; micro-optimizations later, and only if measurement demands them.

After the building blocks, the pattern barely changes. Resolve an address, open a socket, connect, send, receive, close. That loop is the same whether the peer speaks HTTP, SMTP, a binary game protocol, or something you invented this afternoon. What changes in the later stages is almost always the *payload*: is the protocol text or binary, how are messages framed, and how do you parse them safely? HTTP headers, SOCKS5 handshakes, and TLS records look different on the wire, but they all ride the same socket primitives.

`urlget` is a small vehicle for that idea—a mini command-line client you can read end to end. If you finish it and think “that was mostly sockets plus parsing,” the foreword did its job.

— S.F.

Contents

Foreword	1
1 Introduction and Goals	2
1.1 Design Principles	2
1.2 What You Need	2
1.3 How to Read This Tutorial	2

2	Step 1: Basic Project Skeleton	3
2.1	Why This Step Exists	3
2.2	What We Add	3
2.3	New Code	3
2.4	Full Source at This Stage	3
3	Step 2: A Clean <code>usage()</code> Function	4
3.1	Why This Step Exists	4
3.2	What Changes	4
3.3	New Code	4
3.4	Full Source at This Stage	4
4	Step 3: Basic Argument Parsing	5
4.1	Why a Manual Loop?	5
4.2	State We Introduce	5
4.3	New Code — Expanded Usage	5
4.4	New Code — Parsing Loop (core options)	5
4.5	Full Source at This Stage	6
5	Step 4: URL Parsing (<code>parse_url</code>)	9
5.1	Why a Dedicated Struct?	9
5.2	What the Parser Must Handle	9
5.3	Algorithm (high level)	9
5.4	New Code — Core of <code>parse_url</code>	9
5.5	Full Source at This Stage	10
6	Step 5: TCP Connection + Plain HTTP	14
6.1	Why These APIs?	14
6.2	Building the Request	14
6.3	New Code — Resolve, Connect, Request, Read	14
6.4	Full Source at This Stage	15
7	Step 6: Adding HTTPS Support	20
7.1	Why a Helper?	20
7.2	New Code — <code>https_setup</code>	20
7.3	Branching I/O	20
7.4	Full Source at This Stage	21
8	Step 7: Tor / SOCKS5 Support	26
8.1	Why SOCKS5 (not HTTP CONNECT)?	26
8.2	Minimal Handshake	26
8.3	New Code — <code>socks5_connect</code>	26
8.4	Wiring Into <code>main</code>	26
8.5	Full Source at This Stage	27
9	Step 8: Redirect Following	33
9.1	Design Choices	33
9.2	Detecting Redirects	33
9.3	Loop Skeleton	33
9.4	Full Source at This Stage	34

10 Step 9: Advanced Features	40
10.1 Method Selection	40
10.2 Request Construction	40
10.3 Output File (-o)	40
10.4 Basic Authentication (-u) with Base64	40
10.5 Full Source at This Stage	41
11 Step 10: Final Polish and Release Build	48
11.1 Debug Infrastructure	48
11.2 Compile Commands	48
11.3 Feature Map (where everything lives)	49
11.4 Final Complete Source	49
11.5 Closing Thoughts	56

Introduction and Goals

We will build a tool called `urlget`: a small C program that behaves like a minimal subset of `curl`. By the end of this tutorial you will have a single-file client that can:

- Fetch content from `http://` and `https://` URLs
- Optionally route traffic through Tor via SOCKS5
- Follow HTTP redirects (with a configurable limit)
- Send custom headers, POST bodies, and basic authentication
- Write the response body to a file instead of stdout
- Compile clean release builds with debug tracing disabled via `-DNDEBUG`

Design Principles

1. **Incremental.** Each step produces a coherent program. You can stop after any step and still understand what you have.
2. **Readable over clever.** Prefer clear control flow and small helpers over dense one-liners.
3. **POSIX + OpenSSL.** We use standard sockets for TCP and OpenSSL 3 for TLS. No third-party HTTP libraries.
4. **Explain the *why*.** Every new block is introduced with the problem it solves, not only the syntax.

What You Need

- A C compiler (`gcc` or `clang`)
- OpenSSL development headers (for the HTTPS step onward), e.g. `libssl-dev` on Debian/Ubuntu
- Optional: a local Tor daemon listening on `127.0.0.1:9050` if you want to try the SOCKS5 path

How to Read This Tutorial

Each development step has the same shape:

1. A short motivation: what is missing and why we add it now
2. Focused snippets of the **new** code (green background)
3. The **full source so far**: unchanged lines in light gray, newly added lines in green so they stand out in context

The final step assembles the complete `urlget.c` matching a real, usable client.

Step 1: Basic Project Skeleton

Every non-trivial program starts as something almost trivial. Before sockets, TLS, or argument parsing, we need a file that compiles, accepts a URL on the command line, and proves the tool is wired up.

Why This Step Exists

If `main` cannot see `argv`, nothing later matters. A skeleton also gives you a compile command you will reuse for the rest of the tutorial:

```
1 gcc -Wall -Wextra -O2 -o urlget urlget.c
2 ./urlget http://example.com
```

What We Add

- Standard headers for I/O and `EXIT_FAILURE`
- A check that at least one argument (the URL) was provided
- A confirmation print so you can see the argument was received

New Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char *argv[])
5 {
6     if (argc < 2) {
7         fprintf(stderr, "Usage: %s <URL>\n", argv[0]);
8         return EXIT_FAILURE;
9     }
10    printf("Requested: %s\n", argv[1]);
11    return 0;
12 }
```

Error messages go to `stderr`; successful chatter goes to `stdout`. That split stays useful once the program starts printing HTTP bodies to `stdout`.

Full Source at This Stage

Gray = already present. Green-highlighted = added in this step.

```
1 /* urlget.c -- Step 1: basic skeleton */
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(int argc, char *argv[])
6 {
7     if (argc < 2) {
8         fprintf(stderr, "Usage: %s <URL>\n", argv[0]);
9         return EXIT_FAILURE;
10    }
11    printf("Requested: %s\n", argv[1]);
12    return 0;
13 }
```

At this stage everything is new, so the whole file is highlighted. From the next step onward, only the deltas stay green.

Step 2: A Clean `usage()` Function

Hard-coding the usage string inside `main` works once. As soon as we add options, the same message must be printed from many places (missing argument, missing option value, unknown flags). Duplicating the string is how usage text drifts out of date.

Why This Step Exists

- **Single source of truth** for how to invoke the program
- **Immediate exit** on misuse—no half-initialized state
- **static inline** keeps a tiny helper local to this translation unit without the overhead of a separate `.c` file

What Changes

We extract the error path into `usage()`, then call it when the URL is missing. The temporary `printf` confirmation stays for now so you can still smoke-test the binary; it will disappear once we actually fetch pages.

New Code

```
1 static inline void usage(const char *programe)
2 {
3     fprintf(stderr, "Usage: %s <URL>\n", programe);
4     exit(EXIT_FAILURE);
5 }
```

And in `main`:

```
1 if (argc < 2)
2     usage(argv[0]);
```

Passing `argv[0]` means the message shows whatever name the user typed (`./urlget`, `urlget`, a full path, ...).

Full Source at This Stage

Gray = already present. Green-highlighted = added in this step.

```
1 /* urlget.c -- Step 2: usage() helper */
2 #include <stdio.h>
3 #include <stdlib.h>
4 static inline void usage(const char *programe)
5 {
6     fprintf(stderr, "Usage: %s <URL>\n", programe);
7     exit(EXIT_FAILURE);
8 }
9 int main(int argc, char *argv[])
10 {
11     if (argc < 2)
12         usage(argv[0]);
13     printf("Requested: %s\n", argv[1]);
14     return 0;
15 }
```

Step 3: Basic Argument Parsing

Real clients accept flags before the URL. We introduce a straightforward `for` loop over `argv` rather than `getopt_long`, so every branch is visible and easy to extend in later steps.

Why a Manual Loop?

- No extra dependencies or platform quirks
- Optional values (e.g. `-T` with or without a proxy) are easier to express than with classic `getopt`
- Students see exactly when the index advances past an option argument

State We Introduce

- `url_arg` — the single positional URL (required)
- `user_agent` — default browser-like string, overridable with `-A / --user-agent`
- Placeholders for Tor and redirect policy (wired up in later steps)

We also expand `usage()` so it documents the options we claim to support. Empty stubs (`use_tor`, `follow_redirects`, ...) document the roadmap without implementing behavior yet.

New Code — Expanded Usage

```

1 static inline void usage(const char *programe)
2 {
3     fprintf(stderr,
4         "Usage: %s [options] <URL>\n\n"
5         "Options:\n"
6         "  -A, --user-agent STR      Set User-Agent\n"
7         "  -T, --tor [proxy]        Use Tor (default 127.0.0.1:9050)\n"
8         "  -H, --header STR         Add custom header (repeatable)\n"
9         "  -d, --data DATA         POST data (@file supported)\n"
10        "  -o, --output FILE        Write output to file\n"
11        "  -u, --user USER:PASS    Basic authentication\n"
12        "  -I, --head               HEAD request\n"
13        "      --no-location        Do not follow redirects\n"
14        "      --max-redirects N    Max redirects (default: 10)\n",
15        programe);
16    exit(EXIT_FAILURE);
17 }

```

New Code — Parsing Loop (core options)

For this step we fully implement `-A` and recognition of the URL; Tor and the other flags are parsed enough to set variables we will use later.

```

1 const char *user_agent =
2     "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) "
3     "Gecko/20100101 Firefox/128.0";
4 const char *url_arg = NULL;
5
6 for (int i = 1; i < argc; i++) {
7     if (strcmp(argv[i], "-A") == 0 ||
8         strcmp(argv[i], "--user-agent") == 0) {
9         if (i + 1 < argc) user_agent = argv[++i];
10        else usage(argv[0]);
11    }

```

```

12     else if (url_arg == NULL) {
13         url_arg = argv[i];
14     }
15 }
16
17 if (url_arg == NULL)
18     usage(argv[0]);

```

Rule of thumb: option switches that take a value always `++i` after validating `i + 1 < argc`. Anything that is not a recognized option becomes the URL—first one wins.

Full Source at This Stage

Gray = already present. Green-highlighted = added in this step.

```

1 /* urlget.c -- Step 3: argument parsing */
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <stdbool.h>
6
7 static inline void usage(const char *programe)
8 {
9     fprintf(stderr,
10         "Usage: %s [options] <URL>\n\n"
11         "Options:\n"
12         "  -A, --user-agent STR      Set User-Agent\n"
13         "  -T, --tor [proxy]        Use Tor (default 127.0.0.1:9050)\n"
14         "  -H, --header STR         Add custom header (repeatable)\n"
15         "  -d, --data DATA          POST data (@file supported)\n"
16         "  -o, --output FILE         Write output to file\n"
17         "  -u, --user USER:PASS     Basic authentication\n"
18         "  -I, --head                HEAD request\n"
19         "      --no-location         Do not follow redirects\n"
20         "      --max-redirects N     Max redirects (default: 10)\n",
21         programe);
22     exit(EXIT_FAILURE);
23 }
24
25 int main(int argc, char *argv[])
26 {
27     const char *user_agent =
28         "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) "
29         "Gecko/20100101 Firefox/128.0";
30     const char *default_tor = "127.0.0.1:9050";
31
32     const char *url_arg = NULL;
33     const char *tor_proxy = NULL;
34     bool use_tor = false;
35     bool follow_redirects = true;
36     int max_redirects = 10;
37     bool head_request = false;
38     char extra_headers[4096] = {0};
39     char post_data[8192] = {0};
40     char output_file[256] = {0};
41     char auth_header[256] = {0};
42
43     for (int i = 1; i < argc; i++) {
44         if (strcmp(argv[i], "-A") == 0 ||
45             strcmp(argv[i], "--user-agent") == 0) {
46             if (i + 1 < argc) user_agent = argv[++i];
47             else usage(argv[0]);
48         }
49         else if (strcmp(argv[i], "-T") == 0 ||

```

```

49     strcmp(argv[i], "--tor") == 0) {
50         use_tor = true;
51         if (i + 1 < argc && argv[i + 1][0] != '-' &&
52             strchr(argv[i + 1], ':')) {
53             tor_proxy = argv[i + 1];
54             i++;
55         }
56     }
57     else if (strcmp(argv[i], "-H") == 0 ||
58             strcmp(argv[i], "--header") == 0) {
59         if (i + 1 < argc) {
60             strncat(extra_headers, argv[++i],
61                     sizeof(extra_headers) -
62                     strlen(extra_headers) - 3);
63             strcat(extra_headers, "\r\n");
64         } else usage(argv[0]);
65     }
66     else if (strcmp(argv[i], "-d") == 0 ||
67             strcmp(argv[i], "--data") == 0) {
68         if (i + 1 < argc) {
69             const char *val = argv[++i];
70             if (val[0] == '@') {
71                 FILE *f = fopen(val + 1, "r");
72                 if (f) {
73                     fread(post_data, 1,
74                             sizeof(post_data) - 1, f);
75                     fclose(f);
76                 }
77             } else {
78                 strncpy(post_data, val,
79                         sizeof(post_data) - 1);
80             }
81         } else usage(argv[0]);
82     }
83     else if (strcmp(argv[i], "-o") == 0 ||
84             strcmp(argv[i], "--output") == 0) {
85         if (i + 1 < argc)
86             strncpy(output_file, argv[++i],
87                     sizeof(output_file) - 1);
88         else usage(argv[0]);
89     }
90     else if (strcmp(argv[i], "-u") == 0 ||
91             strcmp(argv[i], "--user") == 0) {
92         if (i + 1 < argc)
93             snprintf(auth_header, sizeof(auth_header),
94                     "Authorization: Basic %s\r\n", argv[++i]);
95         else usage(argv[0]);
96     }
97     else if (strcmp(argv[i], "-I") == 0 ||
98             strcmp(argv[i], "--head") == 0) {
99         head_request = true;
100     }
101     else if (strcmp(argv[i], "--no-redirects") == 0) {
102         follow_redirects = false;
103     }
104     else if (strcmp(argv[i], "--max-redirects") == 0 &&
105             i + 1 < argc) {
106         max_redirects = atoi(argv[++i]);
107     }
108     else if (url_arg == NULL) {
109         url_arg = argv[i];
110     }
111 }
112
113 if (url_arg == NULL)

```

```
114     usage(argv[0]);
115     if (use_tor && tor_proxy == NULL)
116         tor_proxy = default_tor;
117
118     printf("URL: %s\n", url_arg);
119     printf("User-Agent: %s\n", user_agent);
120     (void)follow_redirects; (void)max_redirects;
121     (void)head_request; (void)extra_headers;
122     (void)post_data; (void)output_file; (void)auth_header;
123
124     return 0;
125 }
```

The (void) casts silence unused-variable warnings until later steps consume those fields. Parsing early means step 9 will only *use* the flags, not reinvent the CLI.

Step 4: URL Parsing (parse_url)

Sockets need a host and port. HTTP needs a path. The user gives us one string like `https://example.com:8443/`. `parse_url` turns that string into a small structure we can trust for the rest of the program.

Why a Dedicated Struct?

```
1 typedef struct {
2     bool https;
3     char host[256];
4     uint16_t port;
5     char path[512];
6 } url_t;
```

Bundling fields avoids parallel arrays of arguments and makes later redirect handling easy: parse a new URL into a fresh `url_t` and continue.

What the Parser Must Handle

- Optional scheme: `http://` (port 80) or `https://` (port 443)
- Host with optional `:port`
- IPv6 literals in brackets: `[::1]:port`
- Path defaulting to `"/"` when absent
- Query-only forms such as `http://h?x=1` → path `/?x=1`

Algorithm (high level)

1. Copy the argument into a mutable buffer (we will insert `'\0'` while splitting).
2. Detect and strip the scheme; set `https` and the default port.
3. Find the first `/` or `?` — that starts the path (or query). Everything before it is `host[:port]`.
4. Split host and port (special-case `[ipv6]`).
5. Reject oversized components rather than overflowing fixed buffers.

New Code — Core of `parse_url`

```
1 static inline int parse_url(const char *arg, url_t *u)
2 {
3     char buf[1024];
4     if (strlen(arg) >= sizeof(buf)) return -1;
5     strcpy(buf, arg);
6
7     char *p = buf;
8     u->https = false;
9
10    if (strncmp(p, "https://", 8) == 0) {
11        u->https = true; p += 8;
12    } else if (strncmp(p, "http://", 7) == 0) {
13        p += 7;
14    }
15
16    char *sep = strpbrk(p, "/?");
17    char *hostport = p;
```

```

18     if (sep) {
19         if (*sep == '?') {
20             /* "host?query" -> path "/?query" */
21             size_t ql = strlen(sep);
22             if (ql + 2 > sizeof(u->path)) return -1;
23             u->path[0] = '/';
24             memcpy(u->path + 1, sep, ql + 1);
25         } else {
26             if (strlen(sep) >= sizeof(u->path)) return -1;
27             strcpy(u->path, sep);
28         }
29         *sep = '\0';
30     } else {
31         strcpy(u->path, "/");
32     }
33
34     /* host / [ipv6] / port split ... */
35     uint16_t def = u->https ? 443 : 80;
36     u->port = def;
37     /* ... see full source below for host/port details ... */
38     return 0;
39 }
40

```

Call it from main after argument parsing succeeds:

```

1 url_t u = {0};
2 if (parse_url(url_arg, &u) != 0) {
3     fprintf(stderr, "Failed to parse URL\n");
4     return EXIT_FAILURE;
5 }
6 printf("https=%d host=%s port=%u path=%s\n",
7        u.https, u.host, u.port, u.path);

```

Full Source at This Stage

Gray = already present. Green-highlighted = added in this step.

```

1 /* urlget.c -- Step 4: URL parsing */
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <stdbool.h>
6 #include <stdint.h>
7
8 typedef struct {
9     bool https;
10     char host[256];
11     uint16_t port;
12     char path[512];
13 } url_t;
14
15 static inline int parse_url(const char *arg, url_t *u)
16 {
17     char buf[1024];
18     if (strlen(arg) >= sizeof(buf)) return -1;
19     strcpy(buf, arg);
20
21     char *p = buf;
22     u->https = false;
23
24     if (strncmp(p, "https://", 8) == 0) {
25         u->https = true; p += 8;

```

```

26 } else if (strncmp(p, "http://", 7) == 0) {
27     p += 7;
28 }
29
30 char *sep = strpbrk(p, "?");
31 char *hostport = p;
32
33 if (sep) {
34     if (*sep == '?') {
35         size_t ql = strlen(sep);
36         if (ql + 2 > sizeof(u->path)) return -1;
37         u->path[0] = '/';
38         memcpy(u->path + 1, sep, ql + 1);
39     } else {
40         if (strlen(sep) >= sizeof(u->path)) return -1;
41         strcpy(u->path, sep);
42     }
43     *sep = '\0';
44 } else {
45     strcpy(u->path, "/");
46 }
47
48 char *host = u->host;
49 size_t hmax = sizeof(u->host) - 1;
50 uint16_t def = u->https ? 443 : 80;
51 u->port = def;
52
53 if (*hostport == '[') {
54     char *cl = strchr(hostport, ']');
55     if (!cl) return -1;
56     size_t hl = cl - (hostport + 1);
57     if (hl >= hmax) return -1;
58     memcpy(host, hostport + 1, hl);
59     host[hl] = '\0';
60     char *aft = cl + 1;
61     if (*aft == ':')
62         u->port = (uint16_t)strtol(aft + 1, NULL, 10);
63 } else {
64     char *col = strrchr(hostport, ':');
65     if (col) {
66         size_t hl = col - hostport;
67         if (hl >= hmax) return -1;
68         memcpy(host, hostport, hl);
69         host[hl] = '\0';
70         u->port = (uint16_t)strtol(col + 1, NULL, 10);
71     } else {
72         if (strlen(hostport) >= hmax) return -1;
73         strcpy(host, hostport);
74     }
75 }
76
77 if (u->port == 0) u->port = def;
78 return 0;
79 }

```

```

80 static inline void usage(const char *programe)
81 {
82     fprintf(stderr,
83         "Usage: %s [options] <URL>\n\n"
84         "Options:\n"
85         "  -A, --user-agent STR      Set User-Agent\n"
86         "  -T, --tor [proxy]        Use Tor (default 127.0.0.1:9050)\n"
87         "  -H, --header STR         Add custom header (repeatable)\n"
88         "  -d, --data DATA         POST data (@file supported)\n"
89         "  -o, --output FILE        Write output to file\n"

```

```

90     " -u, --user USER:PASS      Basic authentication\n"
91     " -I, --head                 HEAD request\n"
92     "      --no-location          Do not follow redirects\n"
93     "      --max-redirects N      Max redirects (default: 10)\n",
94     progname);
95     exit(EXIT_FAILURE);
96 }
97
98 int main(int argc, char *argv[])
99 {
100     const char *user_agent =
101         "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) "
102         "Gecko/20100101 Firefox/128.0";
103     const char *default_tor = "127.0.0.1:9050";
104
105     const char *url_arg = NULL;
106     const char *tor_proxy = NULL;
107     bool use_tor = false;
108     bool follow_redirects = true;
109     int max_redirects = 10;
110     bool head_request = false;
111     char extra_headers[4096] = {0};
112     char post_data[8192] = {0};
113     char output_file[256] = {0};
114     char auth_header[256] = {0};
115
116     for (int i = 1; i < argc; i++) {
117         if (strcmp(argv[i], "-A") == 0 ||
118             strcmp(argv[i], "--user-agent") == 0) {
119             if (i + 1 < argc) user_agent = argv[++i];
120             else usage(argv[0]);
121         }
122         else if (strcmp(argv[i], "-T") == 0 ||
123                 strcmp(argv[i], "--tor") == 0) {
124             use_tor = true;
125             if (i + 1 < argc && argv[i + 1][0] != '-' &&
126                 strchr(argv[i + 1], ':')) {
127                 tor_proxy = argv[i + 1];
128                 i++;
129             }
130         }
131         else if (strcmp(argv[i], "-H") == 0 ||
132                 strcmp(argv[i], "--header") == 0) {
133             if (i + 1 < argc) {
134                 strncat(extra_headers, argv[++i],
135                     sizeof(extra_headers) -
136                     strlen(extra_headers) - 3);
137                 strcat(extra_headers, "\r\n");
138             } else usage(argv[0]);
139         }
140         else if (strcmp(argv[i], "-d") == 0 ||
141                 strcmp(argv[i], "--data") == 0) {
142             if (i + 1 < argc) {
143                 const char *val = argv[++i];
144                 if (val[0] == '@') {
145                     FILE *f = fopen(val + 1, "r");
146                     if (f) {
147                         fread(post_data, 1,
148                             sizeof(post_data) - 1, f);
149                         fclose(f);
150                     }
151                 } else {
152                     strncpy(post_data, val,
153                         sizeof(post_data) - 1);
154                 }
155             }
156         }
157     }

```

```

155     } else usage(argv[0]);
156 }
157 else if (strcmp(argv[i], "-o") == 0 ||
158         strcmp(argv[i], "--output") == 0) {
159     if (i + 1 < argc)
160         strncpy(output_file, argv[++i],
161                 sizeof(output_file) - 1);
162     else usage(argv[0]);
163 }
164 else if (strcmp(argv[i], "-u") == 0 ||
165         strcmp(argv[i], "--user") == 0) {
166     if (i + 1 < argc)
167         snprintf(auth_header, sizeof(auth_header),
168                  "Authorization: Basic %s\r\n", argv[++i]);
169     else usage(argv[0]);
170 }
171 else if (strcmp(argv[i], "-I") == 0 ||
172         strcmp(argv[i], "--head") == 0) {
173     head_request = true;
174 }
175 else if (strcmp(argv[i], "--no-redirects") == 0) {
176     follow_redirects = false;
177 }
178 else if (strcmp(argv[i], "--max-redirects") == 0 &&
179         i + 1 < argc) {
180     max_redirects = atoi(argv[++i]);
181 }
182 else if (url_arg == NULL) {
183     url_arg = argv[i];
184 }
185 }
186
187 if (url_arg == NULL)
188     usage(argv[0]);
189 if (use_tor && tor_proxy == NULL)
190     tor_proxy = default_tor;
191
192 url_t u = {0};
193 if (parse_url(url_arg, &u) != 0) {
194     fprintf(stderr, "Failed to parse URL\n");
195     return EXIT_FAILURE;
196 }
197 printf("https=%d host=%s port=%u path=%s\n",
198        u.https, u.host, u.port, u.path);
199 (void)user_agent; (void)use_tor; (void)tor_proxy;
200 (void)follow_redirects; (void)max_redirects;
201 (void)head_request; (void)extra_headers;
202 (void)post_data; (void)output_file; (void)auth_header;
203
204 return 0;
205 }

```

Try a few inputs:

```

1 ./urlget http://example.com
2 ./urlget https://example.com:8443/api?x=1
3 ./urlget http://[::1]:8080/status

```

Step 5: TCP Connection + Plain HTTP

With a parsed host, port, and path we can finally speak HTTP. This step stays on cleartext `http://` only; TLS arrives next. The flow is the classic Berkeley sockets path: resolve → socket → connect → write request → read response.

Why These APIs?

- `getaddrinfo` replaces obsolete `gethostbyname` and supports IPv4 and IPv6 transparently (`AF_UNSPEC`).
- Trying every result in the `addrinfo` list handles multi-homed hosts.
- HTTP/1.1 with `Connection: close` keeps the client simple: we read until EOF instead of parsing `Content-Length` or chunked encoding.

Building the Request

A minimal GET looks like:

```
1 GET /path HTTP/1.1\r\n
2 Host: example.com\r\n
3 User-Agent: ...\r\n
4 Accept: */*\r\n
5 Connection: close\r\n
6 \r\n
```

Note the blank line (`\r\n` alone) that terminates the headers. Without it, many servers wait forever for more input.

New Code — Resolve, Connect, Request, Read

```
1 struct addrinfo hints = {0};
2 hints.ai_family = AF_UNSPEC;
3 hints.ai_socktype = SOCK_STREAM;
4 char ps[6];
5 snprintf(ps, sizeof(ps), "%u", u.port);
6
7 struct addrinfo *res = NULL;
8 if (getaddrinfo(u.host, ps, &hints, &res) != 0) {
9     perror("getaddrinfo");
10    return EXIT_FAILURE;
11 }
12
13 int s = -1;
14 for (struct addrinfo *a = res; a; a = a->ai_next) {
15     s = socket(a->ai_family, a->ai_socktype, a->ai_protocol);
16     if (s < 0) continue;
17     if (connect(s, a->ai_addr, a->ai_addrlen) == 0) break;
18     close(s); s = -1;
19 }
20 freeaddrinfo(res);
21 if (s < 0) { perror("connect"); return EXIT_FAILURE; }
22
23 char req[4096];
24 snprintf(req, sizeof(req),
25     "GET %s HTTP/1.1\r\n"
26     "Host: %s\r\n"
27     "User-Agent: %s\r\n"
28     "Accept: */*\r\n"
29     "Connection: close\r\n"
30     "\r\n",
```

```

31     u.path, u.host, user_agent);
32     write(s, req, strlen(req));
33
34     char buf[8192];
35     ssize_t n;
36     while ((n = read(s, buf, sizeof(buf))) > 0)
37         fwrite(buf, 1, n, stdout);
38     close(s);

```

If `u.https` is set we refuse for now—HTTPS needs OpenSSL next.

Full Source at This Stage

Gray = already present. Green-highlighted = added in this step.

```

1  /* urlget.c -- Step 5: TCP + plain HTTP */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <stdbool.h>
6  #include <stdint.h>
7  #include <unistd.h>
8  #include <sys/types.h>
9  #include <sys/socket.h>
10 #include <netdb.h>
11
12 typedef struct {
13     bool https;
14     char host[256];
15     uint16_t port;
16     char path[512];
17 } url_t;
18
19 static inline int parse_url(const char *arg, url_t *u)
20 {
21     char buf[1024];
22     if (strlen(arg) >= sizeof(buf)) return -1;
23     strcpy(buf, arg);
24
25     char *p = buf;
26     u->https = false;
27
28     if (strncmp(p, "https://", 8) == 0) {
29         u->https = true; p += 8;
30     } else if (strncmp(p, "http://", 7) == 0) {
31         p += 7;
32     }
33
34     char *sep = strpbrk(p, "/?");
35     char *hostport = p;
36
37     if (sep) {
38         if (*sep == '?') {
39             size_t ql = strlen(sep);
40             if (ql + 2 > sizeof(u->path)) return -1;
41             u->path[0] = '/';
42             memcpy(u->path + 1, sep, ql + 1);
43         } else {
44             if (strlen(sep) >= sizeof(u->path)) return -1;
45             strcpy(u->path, sep);
46         }
47         *sep = '\0';
48     } else {

```

```

49     strcpy(u->path, "/");
50 }
51
52     char *host = u->host;
53     size_t hmax = sizeof(u->host) - 1;
54     uint16_t def = u->https ? 443 : 80;
55     u->port = def;
56
57     if (*hostport == '[') {
58         char *cl = strchr(hostport, ']');
59         if (!cl) return -1;
60         size_t hl = cl - (hostport + 1);
61         if (hl >= hmax) return -1;
62         memcpy(host, hostport + 1, hl);
63         host[hl] = '\0';
64         char *aft = cl + 1;
65         if (*aft == ':')
66             u->port = (uint16_t)strtol(aft + 1, NULL, 10);
67     } else {
68         char *col = strrchr(hostport, ':');
69         if (col) {
70             size_t hl = col - hostport;
71             if (hl >= hmax) return -1;
72             memcpy(host, hostport, hl);
73             host[hl] = '\0';
74             u->port = (uint16_t)strtol(col + 1, NULL, 10);
75         } else {
76             if (strlen(hostport) >= hmax) return -1;
77             strcpy(host, hostport);
78         }
79     }
80
81     if (u->port == 0) u->port = def;
82     return 0;
83 }
84
85 static inline void usage(const char *programe)
86 {
87     fprintf(stderr,
88         "Usage: %s [options] <URL>\n\n"
89         "Options:\n"
90         "  -A, --user-agent STR      Set User-Agent\n"
91         "  -T, --tor [proxy]         Use Tor (default 127.0.0.1:9050)\n"
92         "  -H, --header STR          Add custom header (repeatable)\n"
93         "  -d, --data DATA          POST data (@file supported)\n"
94         "  -o, --output FILE         Write output to file\n"
95         "  -u, --user USER:PASS     Basic authentication\n"
96         "  -I, --head                HEAD request\n"
97         "      --no-location         Do not follow redirects\n"
98         "      --max-redirects N     Max redirects (default: 10)\n",
99     programe);
100     exit(EXIT_FAILURE);
101 }
102
103 int main(int argc, char *argv[])
104 {
105     const char *user_agent =
106         "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) "
107         "Gecko/20100101 Firefox/128.0";
108     const char *default_tor = "127.0.0.1:9050";
109
110     const char *url_arg = NULL;
111     const char *tor_proxy = NULL;
112     bool use_tor = false;
113     bool follow_redirects = true;

```

```

114     int max_redirects = 10;
115     bool head_request = false;
116     char extra_headers[4096] = {0};
117     char post_data[8192] = {0};
118     char output_file[256] = {0};
119     char auth_header[256] = {0};
120
121     for (int i = 1; i < argc; i++) {
122         if (strcmp(argv[i], "-A") == 0 ||
123             strcmp(argv[i], "--user-agent") == 0) {
124             if (i + 1 < argc) user_agent = argv[++i];
125             else usage(argv[0]);
126         }
127         else if (strcmp(argv[i], "-T") == 0 ||
128                 strcmp(argv[i], "--tor") == 0) {
129             use_tor = true;
130             if (i + 1 < argc && argv[i + 1][0] != '-' &&
131                 strchr(argv[i + 1], ':')) {
132                 tor_proxy = argv[i + 1];
133                 i++;
134             }
135         }
136         else if (strcmp(argv[i], "-H") == 0 ||
137                 strcmp(argv[i], "--header") == 0) {
138             if (i + 1 < argc) {
139                 strncat(extra_headers, argv[++i],
140                         sizeof(extra_headers) -
141                         strlen(extra_headers) - 3);
142                 strcat(extra_headers, "\r\n");
143             } else usage(argv[0]);
144         }
145         else if (strcmp(argv[i], "-d") == 0 ||
146                 strcmp(argv[i], "--data") == 0) {
147             if (i + 1 < argc) {
148                 const char *val = argv[++i];
149                 if (val[0] == '@') {
150                     FILE *f = fopen(val + 1, "r");
151                     if (f) {
152                         fread(post_data, 1,
153                             sizeof(post_data) - 1, f);
154                         fclose(f);
155                     }
156                 } else {
157                     strncpy(post_data, val,
158                             sizeof(post_data) - 1);
159                 }
160             } else usage(argv[0]);
161         }
162         else if (strcmp(argv[i], "-o") == 0 ||
163                 strcmp(argv[i], "--output") == 0) {
164             if (i + 1 < argc)
165                 strncpy(output_file, argv[++i],
166                         sizeof(output_file) - 1);
167             else usage(argv[0]);
168         }
169         else if (strcmp(argv[i], "-u") == 0 ||
170                 strcmp(argv[i], "--user") == 0) {
171             if (i + 1 < argc)
172                 snprintf(auth_header, sizeof(auth_header),
173                          "Authorization: Basic %s\r\n", argv[++i]);
174             else usage(argv[0]);
175         }
176         else if (strcmp(argv[i], "-I") == 0 ||
177                 strcmp(argv[i], "--head") == 0) {
178             head_request = true;

```

```

179     }
180     else if (strcmp(argv[i], "--no-location") == 0) {
181         follow_redirects = false;
182     }
183     else if (strcmp(argv[i], "--max-redirects") == 0 &&
184              i + 1 < argc) {
185         max_redirects = atoi(argv[++i]);
186     }
187     else if (url_arg == NULL) {
188         url_arg = argv[i];
189     }
190 }
191
192 if (url_arg == NULL)
193     usage(argv[0]);
194 if (use_tor && tor_proxy == NULL)
195     tor_proxy = default_tor;
196
197 url_t u = {0};
198 if (parse_url(url_arg, &u) != 0) {
199     fprintf(stderr, "Failed to parse URL\n");
200     return EXIT_FAILURE;
201 }
202
203 if (u.https) {
204     fprintf(stderr, "HTTPS not implemented yet\n");
205     return EXIT_FAILURE;
206 }
207
208 struct addrinfo hints = {0};
209 hints.ai_family = AF_UNSPEC;
210 hints.ai_socktype = SOCK_STREAM;
211 char ps[6];
212 snprintf(ps, sizeof(ps), "%u", u.port);
213
214 struct addrinfo *res = NULL;
215 if (getaddrinfo(u.host, ps, &hints, &res) != 0) {
216     perror("getaddrinfo");
217     return EXIT_FAILURE;
218 }
219
220 int s = -1;
221 for (struct addrinfo *a = res; a; a = a->ai_next) {
222     s = socket(a->ai_family, a->ai_socktype, a->ai_protocol);
223     if (s < 0) continue;
224     if (connect(s, a->ai_addr, a->ai_addrlen) == 0) break;
225     close(s); s = -1;
226 }
227 freeaddrinfo(res);
228 if (s < 0) { perror("connect"); return EXIT_FAILURE; }
229
230 char req[4096];
231 snprintf(req, sizeof(req),
232          "GET %s HTTP/1.1\r\n"
233          "Host: %s\r\n"
234          "User-Agent: %s\r\n"
235          "Accept: */*\r\n"
236          "Connection: close\r\n"
237          "\r\n",
238          u.path, u.host, user_agent);
239 write(s, req, strlen(req));
240
241 char buf[8192];
242 ssize_t n;
243 while ((n = read(s, buf, sizeof(buf))) > 0)

```

```
243     fwrite(buf, 1, n, stdout);
244     close(s);
245
246     (void)use_tor; (void)tor_proxy;
247     (void)follow_redirects; (void)max_redirects;
248     (void)head_request; (void)extra_headers;
249     (void)post_data; (void)output_file; (void)auth_header;
250
251     return 0;
252 }
```

Test with a plain HTTP site (many sites redirect to HTTPS—that is fine; you will still see the redirect response headers and body):

```
1 ./urlget http://example.com/
```

Step 6: Adding HTTPS Support

Most of the modern web is TLS. We wrap the connected TCP socket with OpenSSL so the same HTTP request bytes travel encrypted. Certificate verification is intentionally disabled here for a minimal tutorial client (`SSL_VERIFY_NONE`); production tools should verify the peer.

Why a Helper?

`https_setup` isolates all OpenSSL ceremony:

1. Create a client `SSL_CTX` with `TLS_client_method()`
2. Allocate an SSL object and attach the socket fd
3. Set SNI (`SSL_set_tlsext_host_name`) so virtual hosts work
4. Run the handshake with `SSL_connect`

On success it returns both `ctx` and `ssl` to the caller for later cleanup.

New Code — `https_setup`

```
1 static inline bool https_setup(int sock, const char *hostname,
2                               SSL_CTX **out_ctx, SSL **out_ssl)
3 {
4     SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());
5     if (!ctx) { ERR_print_errors_fp(stderr); return false; }
6
7     SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL);
8
9     SSL *ssl = SSL_new(ctx);
10    if (!ssl) { SSL_CTX_free(ctx); return false; }
11
12    SSL_set_fd(ssl, sock);
13    SSL_set_tlsext_host_name(ssl, hostname);
14
15    if (SSL_connect(ssl) <= 0) {
16        ERR_print_errors_fp(stderr);
17        SSL_free(ssl); SSL_CTX_free(ctx);
18        return false;
19    }
20
21    *out_ctx = ctx;
22    *out_ssl = ssl;
23    return true;
24 }
```

Branching I/O

After connect:

```
1 SSL_CTX *ctx = NULL;
2 SSL *ssl = NULL;
3 if (u.https && !https_setup(s, u.host, &ctx, &ssl)) {
4     close(s);
5     return EXIT_FAILURE;
6 }
7
8 if (u.https)
9     SSL_write(ssl, req, strlen(req));
```

```

10 else
11     write(s, req, strlen(req));
12
13 while ((n = u.https
14         ? SSL_read(ssl, buf, sizeof(buf))
15         : read(s, buf, sizeof(buf))) > 0)
16     fwrite(buf, 1, n, stdout);
17
18 if (u.https) {
19     if (ssl) { SSL_shutdown(ssl); SSL_free(ssl); }
20     if (ctx) SSL_CTX_free(ctx);
21 }
22 close(s);

```

Link with OpenSSL:

```

1 gcc -Wall -Wextra -O2 -o urlget urlget.c -lssl -lcrypto
2 ./urlget https://example.com/

```

Full Source at This Stage

Gray = already present. Green-highlighted = added in this step.

```

1  /* urlget.c -- Step 6: HTTPS via OpenSSL */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <stdbool.h>
6  #include <stdint.h>
7  #include <unistd.h>
8  #include <sys/types.h>
9  #include <sys/socket.h>
10 #include <netdb.h>
11
12 #include <openssl/ssl.h>
13 #include <openssl/err.h>
14
15 typedef struct {
16     bool https;
17     char host[256];
18     uint16_t port;
19     char path[512];
20 } url_t;
21
22 static inline int parse_url(const char *arg, url_t *u)
23 {
24     char buf[1024];
25     if (strlen(arg) >= sizeof(buf)) return -1;
26     strcpy(buf, arg);
27
28     char *p = buf;
29     u->https = false;
30
31     if (strncmp(p, "https://", 8) == 0) {
32         u->https = true; p += 8;
33     } else if (strncmp(p, "http://", 7) == 0) {
34         p += 7;
35     }
36
37     char *sep = strpbrk(p, "/?");
38     char *hostport = p;
39
40     if (sep) {
41         if (*sep == '?') {

```

```

41     size_t ql = strlen(sep);
42     if (ql + 2 > sizeof(u->path)) return -1;
43     u->path[0] = '/';
44     memcpy(u->path + 1, sep, ql + 1);
45 } else {
46     if (strlen(sep) >= sizeof(u->path)) return -1;
47     strcpy(u->path, sep);
48 }
49 *sep = '\0';
50 } else {
51     strcpy(u->path, "/");
52 }
53
54 char *host = u->host;
55 size_t hmax = sizeof(u->host) - 1;
56 uint16_t def = u->https ? 443 : 80;
57 u->port = def;
58
59 if (*hostport == '[') {
60     char *cl = strchr(hostport, ']');
61     if (!cl) return -1;
62     size_t hl = cl - (hostport + 1);
63     if (hl >= hmax) return -1;
64     memcpy(host, hostport + 1, hl);
65     host[hl] = '\0';
66     char *aft = cl + 1;
67     if (*aft == ':')
68         u->port = (uint16_t)strtol(aft + 1, NULL, 10);
69 } else {
70     char *col = strrchr(hostport, ':');
71     if (col) {
72         size_t hl = col - hostport;
73         if (hl >= hmax) return -1;
74         memcpy(host, hostport, hl);
75         host[hl] = '\0';
76         u->port = (uint16_t)strtol(col + 1, NULL, 10);
77     } else {
78         if (strlen(hostport) >= hmax) return -1;
79         strcpy(host, hostport);
80     }
81 }
82
83 if (u->port == 0) u->port = def;
84 return 0;
85 }

```

```

86 static inline bool https_setup(int sock, const char *hostname,
87                               SSL_CTX **out_ctx, SSL **out_ssl)
88 {
89     SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());
90     if (!ctx) { ERR_print_errors_fp(stderr); return false; }
91
92     SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL);
93
94     SSL *ssl = SSL_new(ctx);
95     if (!ssl) { SSL_CTX_free(ctx); return false; }
96
97     SSL_set_fd(ssl, sock);
98     SSL_set_tlsext_host_name(ssl, hostname);
99
100    if (SSL_connect(ssl) <= 0) {
101        ERR_print_errors_fp(stderr);
102        SSL_free(ssl); SSL_CTX_free(ctx);
103        return false;
104    }

```

```

105     *out_ctx = ctx;
106     *out_ssl = ssl;
107     return true;
108 }
109
110 static inline void usage(const char *programe)
111 {
112     fprintf(stderr,
113         "Usage: %s [options] <URL>\n\n"
114         "Options:\n"
115         "  -A, --user-agent STR      Set User-Agent\n"
116         "  -T, --tor [proxy]        Use Tor (default 127.0.0.1:9050)\n"
117         "  -H, --header STR         Add custom header (repeatable)\n"
118         "  -d, --data DATA         POST data (@file supported)\n"
119         "  -o, --output FILE        Write output to file\n"
120         "  -u, --user USER:PASS    Basic authentication\n"
121         "  -I, --head               HEAD request\n"
122         "      --no-location        Do not follow redirects\n"
123         "      --max-redirects N    Max redirects (default: 10)\n",
124         programe);
125     exit(EXIT_FAILURE);
126 }
127
128 int main(int argc, char *argv[])
129 {
130     const char *user_agent =
131         "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) "
132         "Gecko/20100101 Firefox/128.0";
133     const char *default_tor = "127.0.0.1:9050";
134
135     const char *url_arg = NULL;
136     const char *tor_proxy = NULL;
137     bool use_tor = false;
138     bool follow_redirects = true;
139     int max_redirects = 10;
140     bool head_request = false;
141     char extra_headers[4096] = {0};
142     char post_data[8192] = {0};
143     char output_file[256] = {0};
144     char auth_header[256] = {0};
145
146     for (int i = 1; i < argc; i++) {
147         if (strcmp(argv[i], "-A") == 0 ||
148             strcmp(argv[i], "--user-agent") == 0) {
149             if (i + 1 < argc) user_agent = argv[++i];
150             else usage(argv[0]);
151         }
152         else if (strcmp(argv[i], "-T") == 0 ||
153             strcmp(argv[i], "--tor") == 0) {
154             use_tor = true;
155             if (i + 1 < argc && argv[i + 1][0] != '-' &&
156                 strchr(argv[i + 1], ':')) {
157                 tor_proxy = argv[i + 1];
158                 i++;
159             }
160         }
161         else if (strcmp(argv[i], "-H") == 0 ||
162             strcmp(argv[i], "--header") == 0) {
163             if (i + 1 < argc) {
164                 strncat(extra_headers, argv[++i],
165                     sizeof(extra_headers) -
166                     strlen(extra_headers) - 3);
167                 strcat(extra_headers, "\r\n");
168             } else usage(argv[0]);

```

```

169     }
170     else if (strcmp(argv[i], "-d") == 0 ||
171             strcmp(argv[i], "--data") == 0) {
172         if (i + 1 < argc) {
173             const char *val = argv[++i];
174             if (val[0] == '@') {
175                 FILE *f = fopen(val + 1, "r");
176                 if (f) {
177                     fread(post_data, 1,
178                         sizeof(post_data) - 1, f);
179                     fclose(f);
180                 }
181             } else {
182                 strncpy(post_data, val,
183                     sizeof(post_data) - 1);
184             }
185         } else usage(argv[0]);
186     }
187     else if (strcmp(argv[i], "-o") == 0 ||
188             strcmp(argv[i], "--output") == 0) {
189         if (i + 1 < argc)
190             strncpy(output_file, argv[++i],
191                 sizeof(output_file) - 1);
192         else usage(argv[0]);
193     }
194     else if (strcmp(argv[i], "-u") == 0 ||
195             strcmp(argv[i], "--user") == 0) {
196         if (i + 1 < argc)
197             snprintf(auth_header, sizeof(auth_header),
198                 "Authorization: Basic %s\r\n", argv[++i]);
199         else usage(argv[0]);
200     }
201     else if (strcmp(argv[i], "-I") == 0 ||
202             strcmp(argv[i], "--head") == 0) {
203         head_request = true;
204     }
205     else if (strcmp(argv[i], "--no-location") == 0) {
206         follow_redirects = false;
207     }
208     else if (strcmp(argv[i], "--max-redirects") == 0 &&
209             i + 1 < argc) {
210         max_redirects = atoi(argv[++i]);
211     }
212     else if (url_arg == NULL) {
213         url_arg = argv[i];
214     }
215 }
216
217 if (url_arg == NULL)
218     usage(argv[0]);
219 if (use_tor && tor_proxy == NULL)
220     tor_proxy = default_tor;
221
222 url_t u = {0};
223 if (parse_url(url_arg, &u) != 0) {
224     fprintf(stderr, "Failed to parse URL\n");
225     return EXIT_FAILURE;
226 }
227
228 struct addrinfo hints = {0};
229 hints.ai_family = AF_UNSPEC;
230 hints.ai_socktype = SOCK_STREAM;
231 char ps[6];
232 snprintf(ps, sizeof(ps), "%u", u.port);
233

```

```

234     struct addrinfo *res = NULL;
235     if (getaddrinfo(u.host, ps, &hints, &res) != 0) {
236         perror("getaddrinfo");
237         return EXIT_FAILURE;
238     }
239
240     int s = -1;
241     for (struct addrinfo *a = res; a; a = a->ai_next) {
242         s = socket(a->ai_family, a->ai_socktype, a->ai_protocol);
243         if (s < 0) continue;
244         if (connect(s, a->ai_addr, a->ai_addrlen) == 0) break;
245         close(s); s = -1;
246     }
247     freeaddrinfo(res);
248     if (s < 0) { perror("connect"); return EXIT_FAILURE; }
249
250     SSL_CTX *ctx = NULL;
251     SSL *ssl = NULL;
252     if (u.https && !https_setup(s, u.host, &ctx, &ssl)) {
253         close(s);
254         return EXIT_FAILURE;
255     }
256
257     char req[4096];
258     snprintf(req, sizeof(req),
259              "GET %s HTTP/1.1\r\n"
260              "Host: %s\r\n"
261              "User-Agent: %s\r\n"
262              "Accept: */*\r\n"
263              "Connection: close\r\n"
264              "\r\n",
265              u.path, u.host, user_agent);
266
267     if (u.https)
268         SSL_write(ssl, req, strlen(req));
269     else
270         write(s, req, strlen(req));
271
272     char buf[8192];
273     ssize_t n;
274     while ((n = u.https
275             ? SSL_read(ssl, buf, sizeof(buf))
276             : read(s, buf, sizeof(buf))) > 0)
277         fwrite(buf, 1, n, stdout);
278
279     if (u.https) {
280         if (ssl) { SSL_shutdown(ssl); SSL_free(ssl); }
281         if (ctx) SSL_CTX_free(ctx);
282     }
283     close(s);
284
285     (void)use_tor; (void)tor_proxy;
286     (void)follow_redirects; (void)max_redirects;
287     (void)head_request; (void)extra_headers;
288     (void)post_data; (void)output_file; (void)auth_header;
289
290     return 0;
291 }

```

Step 7: Tor / SOCKS5 Support

Direct connections reveal your IP to the destination. With `-T`, we instead open a TCP connection to a local Tor SOCKS5 proxy (default `127.0.0.1:9050`) and ask *it* to reach the real host. The HTTP or HTTPS session then rides on that tunnel.

Why SOCKS5 (not HTTP CONNECT)?

Tor’s default application proxy speaks SOCKS5. Domain names can be sent to the proxy (ATYP = `0x03`), so DNS resolution also happens inside Tor when configured that way—important for anonymity.

Minimal Handshake

1. **Greeting:** version 5, one method “no authentication” (`0x00`).
2. **Server reply:** version 5, method `0x00` accepted.
3. **Connect request:** CMD=CONNECT, address type domain, hostname length + bytes, port in network byte order.
4. **Reply:** REP=`0x00` means success. (We ignore the bind address fields for a minimal client.)

New Code — socks5_connect

```

1 static int socks5_connect(int sock, const char *host, uint16_t port)
2 {
3     unsigned char buf[512];
4     size_t hlen = strlen(host);
5     if (hlen > 255) return -1;
6
7     /* greeting: VER=5, NMETHODS=1, METHOD=0 (no auth) */
8     buf[0] = 0x05; buf[1] = 0x01; buf[2] = 0x00;
9     if (write(sock, buf, 3) != 3) return -1;
10
11     if (read(sock, buf, 2) != 2 || buf[0] != 0x05 || buf[1] != 0x00)
12         return -1;
13
14     /* CONNECT request with domain name */
15     buf[0] = 0x05; buf[1] = 0x01; buf[2] = 0x00; buf[3] = 0x03;
16     buf[4] = (unsigned char)hlen;
17     memcpy(buf + 5, host, hlen);
18     buf[5 + hlen] = port >> 8;
19     buf[6 + hlen] = port & 0xff;
20
21     if (write(sock, buf, 7 + hlen) != (ssize_t)(7 + hlen)) return -1;
22     if (read(sock, buf, 4) != 4 || buf[1] != 0x00) return -1;
23     return 0;
24 }
```

Wiring Into main

When Tor is enabled, DNS/connect targets the *proxy*, not the origin. After the TCP connect succeeds, run the SOCKS5 handshake with the real host and port from `url_t`. TLS (if any) still uses the real hostname for SNI.

```

1 const char *connect_host = use_tor ? tor_proxy : u.host;
2 uint16_t connect_port = use_tor ? 9050 : u.port;
3
```

```

4  if (use_tor) {
5      char pbuf[256];
6      strncpy(pbuf, tor_proxy, sizeof(pbuf) - 1);
7      char *colon = strrchr(pbuf, ':');
8      if (colon) {
9          *colon = '\0';
10         connect_port = (uint16_t)strtol(colon + 1, NULL, 10);
11         connect_host = pbuf;
12     }
13 }
14 /* getaddrinfo(connect_host) + connect ... */
15 if (use_tor && socks5_connect(s, u.host, u.port) != 0) {
16     close(s);
17     return EXIT_FAILURE;
18 }

```

Full Source at This Stage

Gray = already present. Green-highlighted = added in this step.

```

1  /* urlget.c -- Step 7: Tor / SOCKS5 */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <stdbool.h>
6  #include <stdint.h>
7  #include <unistd.h>
8  #include <sys/types.h>
9  #include <sys/socket.h>
10 #include <netdb.h>
11 #include <openssl/ssl.h>
12 #include <openssl/err.h>
13
14 typedef struct {
15     bool https;
16     char host[256];
17     uint16_t port;
18     char path[512];
19 } url_t;
20
21 static inline int parse_url(const char *arg, url_t *u)
22 {
23     char buf[1024];
24     if (strlen(arg) >= sizeof(buf)) return -1;
25     strcpy(buf, arg);
26     char *p = buf;
27     u->https = false;
28     if (strncmp(p, "https://", 8) == 0) { u->https = true; p += 8; }
29     else if (strncmp(p, "http://", 7) == 0) { p += 7; }
30     char *sep = strpbrk(p, "/?");
31     char *hostport = p;
32     if (sep) {
33         if (*sep == '?') {
34             size_t ql = strlen(sep);
35             if (ql + 2 > sizeof(u->path)) return -1;
36             u->path[0] = '/';
37             memcpy(u->path + 1, sep, ql + 1);
38         } else {
39             if (strlen(sep) >= sizeof(u->path)) return -1;
40             strcpy(u->path, sep);
41         }
42         *sep = '\0';
43     } else strcpy(u->path, "/");
44     char *host = u->host;

```

```

45     size_t hmax = sizeof(u->host) - 1;
46     uint16_t def = u->https ? 443 : 80;
47     u->port = def;
48     if (*hostport == '[') {
49         char *cl = strchr(hostport, ']');
50         if (!cl) return -1;
51         size_t hl = cl - (hostport + 1);
52         if (hl >= hmax) return -1;
53         memcpy(host, hostport + 1, hl);
54         host[hl] = '\0';
55         if (*(cl + 1) == ':')
56             u->port = (uint16_t)strtol(cl + 2, NULL, 10);
57     } else {
58         char *col = strrchr(hostport, ':');
59         if (col) {
60             size_t hl = col - hostport;
61             if (hl >= hmax) return -1;
62             memcpy(host, hostport, hl);
63             host[hl] = '\0';
64             u->port = (uint16_t)strtol(col + 1, NULL, 10);
65         } else {
66             if (strlen(hostport) >= hmax) return -1;
67             strcpy(host, hostport);
68         }
69     }
70     if (u->port == 0) u->port = def;
71     return 0;
72 }

73 static int socks5_connect(int sock, const char *host, uint16_t port)
74 {
75     unsigned char buf[512];
76     size_t hlen = strlen(host);
77     if (hlen > 255) return -1;
78
79     buf[0] = 0x05; buf[1] = 0x01; buf[2] = 0x00;
80     if (write(sock, buf, 3) != 3) return -1;
81     if (read(sock, buf, 2) != 2 || buf[0] != 0x05 || buf[1] != 0x00)
82         return -1;
83
84     buf[0] = 0x05; buf[1] = 0x01; buf[2] = 0x00; buf[3] = 0x03;
85     buf[4] = (unsigned char)hlen;
86     memcpy(buf + 5, host, hlen);
87     buf[5 + hlen] = port >> 8;
88     buf[6 + hlen] = port & 0xff;
89     if (write(sock, buf, 7 + hlen) != (ssize_t)(7 + hlen)) return -1;
90     if (read(sock, buf, 4) != 4 || buf[1] != 0x00) return -1;
91     return 0;
92 }

93 static inline bool https_setup(int sock, const char *hostname,
94                                SSL_CTX **out_ctx, SSL **out_ssl)
95 {
96     SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());
97     if (!ctx) { ERR_print_errors_fp(stderr); return false; }
98     SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL);
99     SSL *ssl = SSL_new(ctx);
100    if (!ssl) { SSL_CTX_free(ctx); return false; }
101    SSL_set_fd(ssl, sock);
102    SSL_set_tlsext_host_name(ssl, hostname);
103    if (SSL_connect(ssl) <= 0) {
104        ERR_print_errors_fp(stderr);
105        SSL_free(ssl); SSL_CTX_free(ctx);
106        return false;
107    }
108    *out_ctx = ctx;

```

```

109     *out_ssl = ssl;
110     return true;
111 }
112
113 static inline void usage(const char *progrname)
114 {
115     fprintf(stderr,
116         "Usage: %s [options] <URL>\n\n"
117         "Options:\n"
118         "  -A, --user-agent STR      Set User-Agent\n"
119         "  -T, --tor [proxy]         Use Tor (default 127.0.0.1:9050)\n"
120         "  -H, --header STR          Add custom header (repeatable)\n"
121         "  -d, --data DATA          POST data (@file supported)\n"
122         "  -o, --output FILE          Write output to file\n"
123         "  -u, --user USER:PASS      Basic authentication\n"
124         "  -I, --head                 HEAD request\n"
125         "      --no-location          Do not follow redirects\n"
126         "      --max-redirects N      Max redirects (default: 10)\n",
127         progrname);
128     exit(EXIT_FAILURE);
129 }
130
131 int main(int argc, char *argv[])
132 {
133     const char *user_agent =
134         "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) "
135         "Gecko/20100101 Firefox/128.0";
136     const char *default_tor = "127.0.0.1:9050";
137     const char *url_arg = NULL;
138     const char *tor_proxy = NULL;
139     bool use_tor = false;
140     bool follow_redirects = true;
141     int max_redirects = 10;
142     bool head_request = false;
143     char extra_headers[4096] = {0};
144     char post_data[8192] = {0};
145     char output_file[256] = {0};
146     char auth_header[256] = {0};
147
148     for (int i = 1; i < argc; i++) {
149         if (strcmp(argv[i], "-A") == 0 ||
150             strcmp(argv[i], "--user-agent") == 0) {
151             if (i + 1 < argc) user_agent = argv[++i];
152             else usage(argv[0]);
153         }
154         else if (strcmp(argv[i], "-T") == 0 ||
155                 strcmp(argv[i], "--tor") == 0) {
156             use_tor = true;
157             if (i + 1 < argc && argv[i + 1][0] != '-' &&
158                 strchr(argv[i + 1], ':')) {
159                 tor_proxy = argv[i + 1];
160                 i++;
161             }
162         }
163         else if (strcmp(argv[i], "-H") == 0 ||
164                 strcmp(argv[i], "--header") == 0) {
165             if (i + 1 < argc) {
166                 strncat(extra_headers, argv[++i],
167                     sizeof(extra_headers) -
168                     strlen(extra_headers) - 3);
169                 strcat(extra_headers, "\r\n");
170             } else usage(argv[0]);
171         }
172         else if (strcmp(argv[i], "-d") == 0 ||
173                 strcmp(argv[i], "--data") == 0) {

```

```

174         if (i + 1 < argc) {
175             const char *val = argv[++i];
176             if (val[0] == '@') {
177                 FILE *f = fopen(val + 1, "r");
178                 if (f) {
179                     fread(post_data, 1,
180                         sizeof(post_data) - 1, f);
181                     fclose(f);
182                 }
183             } else
184                 strncpy(post_data, val, sizeof(post_data) - 1);
185         } else usage(argv[0]);
186     }
187     else if (strcmp(argv[i], "-o") == 0 ||
188             strcmp(argv[i], "--output") == 0) {
189         if (i + 1 < argc)
190             strncpy(output_file, argv[++i],
191                 sizeof(output_file) - 1);
192         else usage(argv[0]);
193     }
194     else if (strcmp(argv[i], "-u") == 0 ||
195             strcmp(argv[i], "--user") == 0) {
196         if (i + 1 < argc)
197             snprintf(auth_header, sizeof(auth_header),
198                 "Authorization: Basic %s\r\n", argv[++i]);
199         else usage(argv[0]);
200     }
201     else if (strcmp(argv[i], "-I") == 0 ||
202             strcmp(argv[i], "--head") == 0)
203         head_request = true;
204     else if (strcmp(argv[i], "--no-location") == 0)
205         follow_redirects = false;
206     else if (strcmp(argv[i], "--max-redirects") == 0 &&
207             i + 1 < argc)
208         max_redirects = atoi(argv[++i]);
209     else if (url_arg == NULL)
210         url_arg = argv[i];
211 }
212
213 if (url_arg == NULL) usage(argv[0]);
214 if (use_tor && tor_proxy == NULL) tor_proxy = default_tor;
215
216 url_t u = {0};
217 if (parse_url(url_arg, &u) != 0) {
218     fprintf(stderr, "Failed to parse URL\n");
219     return EXIT_FAILURE;
220 }
221
222 const char *connect_host = use_tor ? tor_proxy : u.host;
223 uint16_t connect_port = use_tor ? 9050 : u.port;
224 char pbuf[256];
225 if (use_tor) {
226     strncpy(pbuf, tor_proxy, sizeof(pbuf) - 1);
227     char *colon = strrchr(pbuf, ':');
228     if (colon) {
229         *colon = '\0';
230         connect_port = (uint16_t)strtol(colon + 1, NULL, 10);
231         connect_host = pbuf;
232     }
233 }
234
235 struct addrinfo hints = {0};
236 hints.ai_family = AF_UNSPEC;
237 hints.ai_socktype = SOCK_STREAM;
238 char ps[6];

```

```

237     snprintf(ps, sizeof(ps), "%u", connect_port);
238
239     struct addrinfo *res = NULL;
240     if (getaddrinfo(connect_host, ps, &hints, &res) != 0) {
241         perror("getaddrinfo");
242         return EXIT_FAILURE;
243     }
244
245     int s = -1;
246     for (struct addrinfo *a = res; a; a = a->ai_next) {
247         s = socket(a->ai_family, a->ai_socktype, a->ai_protocol);
248         if (s < 0) continue;
249         if (connect(s, a->ai_addr, a->ai_addrlen) == 0) break;
250         close(s); s = -1;
251     }
252     freeaddrinfo(res);
253     if (s < 0) { perror("connect"); return EXIT_FAILURE; }
254
255     if (use_tor && socks5_connect(s, u.host, u.port) != 0) {
256         close(s);
257         return EXIT_FAILURE;
258     }
259
260     SSL_CTX *ctx = NULL;
261     SSL *ssl = NULL;
262     if (u.https && !https_setup(s, u.host, &ctx, &ssl)) {
263         close(s);
264         return EXIT_FAILURE;
265     }
266
267     char req[4096];
268     snprintf(req, sizeof(req),
269             "GET %s HTTP/1.1\r\n"
270             "Host: %s\r\n"
271             "User-Agent: %s\r\n"
272             "Accept: */*\r\n"
273             "Connection: close\r\n"
274             "\r\n",
275             u.path, u.host, user_agent);
276
277     if (u.https)
278         SSL_write(ssl, req, strlen(req));
279     else
280         write(s, req, strlen(req));
281
282     char buf[8192];
283     ssize_t n;
284     while ((n = u.https
285             ? SSL_read(ssl, buf, sizeof(buf))
286             : read(s, buf, sizeof(buf))) > 0)
287         fwrite(buf, 1, n, stdout);
288
289     if (u.https) {
290         if (ssl) { SSL_shutdown(ssl); SSL_free(ssl); }
291         if (ctx) SSL_CTX_free(ctx);
292     }
293     close(s);
294     (void)follow_redirects; (void)max_redirects;
295     (void)head_request; (void)extra_headers;
296     (void)post_data; (void)output_file; (void)auth_header;
297     return 0;

```

Requires a running Tor daemon (or any SOCKS5 proxy) on the configured address:

```
./urlget -T https://check.torproject.org/
```

```
2 ./urlget -T 127.0.0.1:9050 https://example.com/
```

Step 8: Redirect Following

Many URLs answer with 301/302/307 and a `Location:` header. A usable client should re-issue the request at the new URL (unless the user disables that). We wrap the fetch path in a loop controlled by `follow_redirects` and `max_redirects`.

Design Choices

- Keep a mutable `current_url` buffer; each redirect overwrites it and we `continue` the loop.
- Re-run `parse_url`, DNS/proxy connect, TLS, and the HTTP exchange for every hop—simpler than special-casing relative locations in this minimal tool (absolute `Location` URLs work best).
- Cap hops (default 10) to avoid infinite redirect loops.
- `--no-location` sets `follow_redirects = false`.

Detecting Redirects

After reading the response into `buf`, a simple approach (used by our final program) scans for the substring `Location:`. This is not a full HTTP parser—it is good enough for a teaching client when the header appears in the first read chunk.

```

1  if (follow_redirects && strstr(buf, "Location:") &&
2      redirect_count < max_redirects) {
3      char location[1024] = {0};
4      char *loc = strstr(buf, "Location:");
5      if (loc) sscanf(loc, "Location: %1023s", location);
6      if (location[0]) {
7          redirect_count++;
8          strncpy(current_url, location, sizeof(current_url) - 1);
9          continue; /* retry outer while with new URL */
10     }
11 }
12 break; /* done */

```

Loop Skeleton

```

1  int redirect_count = 0;
2  char current_url[1024];
3  strncpy(current_url, url_arg, sizeof(current_url) - 1);
4
5  while (redirect_count <= max_redirects) {
6      url_t u = {0};
7      if (parse_url(current_url, &u) != 0)
8          return EXIT_FAILURE;
9
10     /* connect (+ optional SOCKS5) ... */
11     /* optional HTTPS ... */
12     /* send request, read body into buf / stdout ... */
13
14     if (/* Location found and following enabled */)
15         continue;
16     break;
17 }

```

Full Source at This Stage

Gray = already present. Green-highlighted = added in this step.

```

1  /* urlget.c -- Step 8: redirect following */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <stdbool.h>
6  #include <stdint.h>
7  #include <unistd.h>
8  #include <sys/types.h>
9  #include <sys/socket.h>
10 #include <netdb.h>
11 #include <openssl/ssl.h>
12 #include <openssl/err.h>
13
14 typedef struct {
15     bool https;
16     char host[256];
17     uint16_t port;
18     char path[512];
19 } url_t;
20
21 static inline int parse_url(const char *arg, url_t *u)
22 {
23     char buf[1024];
24     if (strlen(arg) >= sizeof(buf)) return -1;
25     strcpy(buf, arg);
26     char *p = buf;
27     u->https = false;
28     if (strncmp(p, "https://", 8) == 0) { u->https = true; p += 8; }
29     else if (strncmp(p, "http://", 7) == 0) { p += 7; }
30     char *sep = strpbrk(p, "/?");
31     char *hostport = p;
32     if (sep) {
33         if (*sep == '?') {
34             size_t ql = strlen(sep);
35             if (ql + 2 > sizeof(u->path)) return -1;
36             u->path[0] = '/';
37             memcpy(u->path + 1, sep, ql + 1);
38         } else {
39             if (strlen(sep) >= sizeof(u->path)) return -1;
40             strcpy(u->path, sep);
41         }
42         *sep = '\0';
43     } else strcpy(u->path, "/");
44     char *host = u->host;
45     size_t hmax = sizeof(u->host) - 1;
46     uint16_t def = u->https ? 443 : 80;
47     u->port = def;
48     if (*hostport == '[') {
49         char *cl = strchr(hostport, ']');
50         if (!cl) return -1;
51         size_t hl = cl - (hostport + 1);
52         if (hl >= hmax) return -1;
53         memcpy(host, hostport + 1, hl);
54         host[hl] = '\0';
55         if (*(cl + 1) == ':')
56             u->port = (uint16_t)strtol(cl + 2, NULL, 10);
57     } else {
58         char *col = strrchr(hostport, ':');
59         if (col) {
60             size_t hl = col - hostport;
61             if (hl >= hmax) return -1;

```

```

62     memcpy(host, hostport, hl);
63     host[hl] = '\0';
64     u->port = (uint16_t)strtol(col + 1, NULL, 10);
65 } else {
66     if (strlen(hostport) >= hmax) return -1;
67     strcpy(host, hostport);
68 }
69 }
70 if (u->port == 0) u->port = def;
71 return 0;
72 }
73
74 static int socks5_connect(int sock, const char *host, uint16_t port)
75 {
76     unsigned char buf[512];
77     size_t hlen = strlen(host);
78     if (hlen > 255) return -1;
79     buf[0] = 0x05; buf[1] = 0x01; buf[2] = 0x00;
80     if (write(sock, buf, 3) != 3) return -1;
81     if (read(sock, buf, 2) != 2 || buf[0] != 0x05 || buf[1] != 0x00)
82         return -1;
83     buf[0] = 0x05; buf[1] = 0x01; buf[2] = 0x00; buf[3] = 0x03;
84     buf[4] = (unsigned char)hlen;
85     memcpy(buf + 5, host, hlen);
86     buf[5 + hlen] = port >> 8;
87     buf[6 + hlen] = port & 0xff;
88     if (write(sock, buf, 7 + hlen) != (ssize_t)(7 + hlen)) return -1;
89     if (read(sock, buf, 4) != 4 || buf[1] != 0x00) return -1;
90     return 0;
91 }
92
93 static inline bool https_setup(int sock, const char *hostname,
94                               SSL_CTX **out_ctx, SSL **out_ssl)
95 {
96     SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());
97     if (!ctx) { ERR_print_errors_fp(stderr); return false; }
98     SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL);
99     SSL *ssl = SSL_new(ctx);
100    if (!ssl) { SSL_CTX_free(ctx); return false; }
101    SSL_set_fd(ssl, sock);
102    SSL_set_tlsext_host_name(ssl, hostname);
103    if (SSL_connect(ssl) <= 0) {
104        ERR_print_errors_fp(stderr);
105        SSL_free(ssl); SSL_CTX_free(ctx);
106        return false;
107    }
108    *out_ctx = ctx;
109    *out_ssl = ssl;
110    return true;
111 }
112
113 static inline void usage(const char *progname)
114 {
115     fprintf(stderr,
116         "Usage: %s [options] <URL>\n\n"
117         "Options:\n"
118         "  -A, --user-agent STR      Set User-Agent\n"
119         "  -T, --tor [proxy]         Use Tor (default 127.0.0.1:9050)\n"
120         "  -H, --header STR          Add custom header (repeatable)\n"
121         "  -d, --data DATA          POST data (@file supported)\n"
122         "  -o, --output FILE         Write output to file\n"
123         "  -u, --user USER:PASS     Basic authentication\n"
124         "  -I, --head                HEAD request\n"
125         "      --no-location         Do not follow redirects\n"
126         "      --max-redirects N     Max redirects (default: 10)\n",

```

```

127     progname);
128     exit(EXIT_FAILURE);
129 }
130
131 int main(int argc, char *argv[])
132 {
133     const char *user_agent =
134         "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) "
135         "Gecko/20100101 Firefox/128.0";
136     const char *default_tor = "127.0.0.1:9050";
137     const char *url_arg = NULL;
138     const char *tor_proxy = NULL;
139     bool use_tor = false;
140     bool follow_redirects = true;
141     int max_redirects = 10;
142     bool head_request = false;
143     char extra_headers[4096] = {0};
144     char post_data[8192] = {0};
145     char output_file[256] = {0};
146     char auth_header[256] = {0};
147
148     for (int i = 1; i < argc; i++) {
149         if (strcmp(argv[i], "-A") == 0 ||
150             strcmp(argv[i], "--user-agent") == 0) {
151             if (i + 1 < argc) user_agent = argv[++i];
152             else usage(argv[0]);
153         }
154         else if (strcmp(argv[i], "-T") == 0 ||
155                 strcmp(argv[i], "--tor") == 0) {
156             use_tor = true;
157             if (i + 1 < argc && argv[i + 1][0] != '-' &&
158                 strchr(argv[i + 1], ':')) {
159                 tor_proxy = argv[i + 1];
160                 i++;
161             }
162         }
163         else if (strcmp(argv[i], "-H") == 0 ||
164                 strcmp(argv[i], "--header") == 0) {
165             if (i + 1 < argc) {
166                 strncat(extra_headers, argv[++i],
167                     sizeof(extra_headers) -
168                     strlen(extra_headers) - 3);
169                 strcat(extra_headers, "\r\n");
170             } else usage(argv[0]);
171         }
172         else if (strcmp(argv[i], "-d") == 0 ||
173                 strcmp(argv[i], "--data") == 0) {
174             if (i + 1 < argc) {
175                 const char *val = argv[++i];
176                 if (val[0] == '@') {
177                     FILE *f = fopen(val + 1, "r");
178                     if (f) {
179                         fread(post_data, 1,
180                             sizeof(post_data) - 1, f);
181                         fclose(f);
182                     }
183                 } else
184                     strncpy(post_data, val, sizeof(post_data) - 1);
185             } else usage(argv[0]);
186         }
187         else if (strcmp(argv[i], "-o") == 0 ||
188                 strcmp(argv[i], "--output") == 0) {
189             if (i + 1 < argc)
190                 strncpy(output_file, argv[++i],
191                     sizeof(output_file) - 1);

```

```

192     else usage(argv[0]);
193 }
194 else if (strcmp(argv[i], "-u") == 0 ||
195         strcmp(argv[i], "--user") == 0) {
196     if (i + 1 < argc)
197         snprintf(auth_header, sizeof(auth_header),
198                 "Authorization: Basic %s\r\n", argv[++i]);
199     else usage(argv[0]);
200 }
201 else if (strcmp(argv[i], "-I") == 0 ||
202         strcmp(argv[i], "--head") == 0)
203     head_request = true;
204 else if (strcmp(argv[i], "--no-location") == 0)
205     follow_redirects = false;
206 else if (strcmp(argv[i], "--max-redirects") == 0 &&
207         i + 1 < argc)
208     max_redirects = atoi(argv[++i]);
209 else if (url_arg == NULL)
210     url_arg = argv[i];
211 }
212
213 if (url_arg == NULL) usage(argv[0]);
214 if (use_tor && tor_proxy == NULL) tor_proxy = default_tor;
215
216 int redirect_count = 0;
217 char current_url[1024];
218 strncpy(current_url, url_arg, sizeof(current_url) - 1);
219
220 while (redirect_count <= max_redirects) {
221     url_t u = {0};
222     if (parse_url(current_url, &u) != 0) {
223         fprintf(stderr, "Failed to parse URL\n");
224         return EXIT_FAILURE;
225     }
226
227     const char *connect_host = use_tor ? tor_proxy : u.host;
228     uint16_t connect_port = use_tor ? 9050 : u.port;
229     char pbuf[256];
230     if (use_tor) {
231         strncpy(pbuf, tor_proxy, sizeof(pbuf) - 1);
232         char *colon = strrchr(pbuf, ':');
233         if (colon) {
234             *colon = '\0';
235             connect_port =
236                 (uint16_t)strtol(colon + 1, NULL, 10);
237             connect_host = pbuf;
238         }
239     }
240
241     struct addrinfo hints = {0};
242     hints.ai_family = AF_UNSPEC;
243     hints.ai_socktype = SOCK_STREAM;
244     char ps[6];
245     snprintf(ps, sizeof(ps), "%u", connect_port);
246
247     struct addrinfo *res = NULL;
248     if (getaddrinfo(connect_host, ps, &hints, &res) != 0) {
249         perror("getaddrinfo");
250         return EXIT_FAILURE;
251     }
252
253     int s = -1;
254     for (struct addrinfo *a = res; a; a = a->ai_next) {
255         s = socket(a->ai_family, a->ai_socktype, a->ai_protocol);
256         if (s < 0) continue;

```

```

256         if (connect(s, a->ai_addr, a->ai_addrlen) == 0) break;
257         close(s); s = -1;
258     }
259     freeaddrinfo(res);
260     if (s < 0) { perror("connect"); return EXIT_FAILURE; }
261
262     if (use_tor && socks5_connect(s, u.host, u.port) != 0) {
263         close(s);
264         return EXIT_FAILURE;
265     }
266
267     SSL_CTX *ctx = NULL;
268     SSL *ssl = NULL;
269     if (u.https && !https_setup(s, u.host, &ctx, &ssl)) {
270         close(s);
271         return EXIT_FAILURE;
272     }
273
274     char req[4096];
275     snprintf(req, sizeof(req),
276              "GET %s HTTP/1.1\r\n"
277              "Host: %s\r\n"
278              "User-Agent: %s\r\n"
279              "Accept: */*\r\n"
280              "Connection: close\r\n"
281              "\r\n",
282              u.path, u.host, user_agent);
283
284     if (u.https)
285         SSL_write(ssl, req, strlen(req));
286     else
287         write(s, req, strlen(req));
288
289     char buf[8192];
290     ssize_t n;
291     while ((n = u.https
292             ? SSL_read(ssl, buf, sizeof(buf))
293             : read(s, buf, sizeof(buf))) > 0)
294         fwrite(buf, 1, n, stdout);
295
296     if (u.https) {
297         if (ssl) { SSL_shutdown(ssl); SSL_free(ssl); }
298         if (ctx) SSL_CTX_free(ctx);
299     }
300     close(s);
301
302     if (follow_redirects && strstr(buf, "Location:") &&
303         redirect_count < max_redirects) {
304         char location[1024] = {0};
305         char *loc = strstr(buf, "Location:");
306         if (loc) sscanf(loc, "Location: %1023s", location);
307         if (location[0]) {
308             redirect_count++;
309             strncpy(current_url, location,
310                     sizeof(current_url) - 1);
311             continue;
312         }
313     }
314     break;
315 }
316
317 (void)head_request; (void)extra_headers;
318 (void)post_data; (void)output_file; (void)auth_header;
319
320 return 0;

```

```
320 }
```

```
1 ./urlget http://example.com/          # often 301 -> https
2 ./urlget --no-location http://example.com/
3 ./urlget --max-redirects 2 http://example.com/
```

Step 9: Advanced Features

Argument parsing already collected values for custom headers, POST bodies, output files, basic auth, and HEAD requests. This step *uses* those values when building and sending the HTTP message, and when writing the response.

Method Selection

```
1 const char *method = head_request ? "HEAD"
2       : (post_data[0] ? "POST" : "GET");
```

Priority: explicit `-I/--head` wins; otherwise any `-d` data implies POST; default remains GET.

Request Construction

Optional header blocks are injected as pre-formatted strings ending in `\r\n` (see the argument parser):

```
1 int len = snprintf(req, sizeof(req),
2     "%s %s HTTP/1.1\r\n"
3     "Host: %s\r\n"
4     "User-Agent: %s\r\n"
5     "%s" /* extra_headers */
6     "%s" /* auth_header */
7     "Accept: */*\r\n"
8     "Connection: close\r\n",
9     method, u.path, u.host, user_agent,
10    extra_headers, auth_header);
11
12 if (post_data[0]) {
13     len += snprintf(req + len, sizeof(req) - len,
14         "Content-Length: %zu\r\n\r\n%s",
15         strlen(post_data), post_data);
16 } else {
17     strcat(req + len, "\r\n");
18 }
```

Output File (-o)

```
1 FILE *out = stdout;
2 if (output_file[0]) {
3     out = fopen(output_file, "w");
4     if (!out) { perror("fopen"); return EXIT_FAILURE; }
5 }
6 /* ... fwrite(buf, 1, n, out); ... */
7 if (out != stdout) fclose(out);
```

Basic Authentication (-u) with Base64

HTTP Basic auth expects `Authorization: Basic <base64(user:pass)>`. We accept curl-style `USER:PASS` and encode it ourselves with a small RFC 4648 helper—no external process, no extra libraries.

```
1 static inline int base64_encode(const void *data, size_t len,
2                                char *out, size_t out_sz)
3 {
4     static const char T[] =
5         "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"
```

```

6         "0123456789+/";
7     const unsigned char *s = data;
8     size_t o = 0;
9
10    for (size_t i = 0; i < len; i += 3) {
11        unsigned v = (unsigned)s[i] << 16;
12        int n = 1;
13        if (i + 1 < len) { v |= (unsigned)s[i + 1] << 8; n = 2; }
14        if (i + 2 < len) { v |= (unsigned)s[i + 2]; n = 3; }
15        if (o + 4 >= out_sz) return -1;
16        out[o++] = T[(v >> 18) & 63];
17        out[o++] = T[(v >> 12) & 63];
18        out[o++] = (n >= 2) ? T[(v >> 6) & 63] : '=';
19        out[o++] = (n >= 3) ? T[v & 63] : '=';
20    }
21    if (o >= out_sz) return -1;
22    out[o] = '\0';
23    return (int)o;
24 }

```

When parsing `-u / --user`:

```

1 char b64[200];
2 const char *cred = argv[++i];
3 if (base64_encode(cred, strlen(cred), b64, sizeof(b64)) < 0) {
4     fprintf(stderr, "credentials too long\n");
5     return EXIT_FAILURE;
6 }
7 snprintf(auth_header, sizeof(auth_header),
8         "Authorization: Basic %s\r\n", b64);

```

The encoder walks the input three bytes at a time, emits four alphabet characters, and writes = padding when the final group is short. That is enough for credential strings and keeps the whole client dependency-free beyond OpenSSL (which we already link for HTTPS).

Full Source at This Stage

Gray = already present. Green-highlighted = added in this step.

```

1 /* urlget.c -- Step 9: advanced request features */
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <stdbool.h>
6 #include <stdint.h>
7 #include <unistd.h>
8 #include <sys/types.h>
9 #include <sys/socket.h>
10 #include <netdb.h>
11 #include <openssl/ssl.h>
12 #include <openssl/err.h>
13
14 /* RFC 4648 Base64 for HTTP Basic auth (-u USER:PASS). */
15 static inline int base64_encode(const void *data, size_t len,
16                                char *out, size_t out_sz)
17 {
18     static const char T[] =
19         "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"
20         "0123456789+/";
21     const unsigned char *s = data;
22     size_t o = 0;
23
24     for (size_t i = 0; i < len; i += 3) {
25         unsigned v = (unsigned)s[i] << 16;

```

```

25     int n = 1;
26     if (i + 1 < len) { v |= (unsigned)s[i + 1] << 8; n = 2; }
27     if (i + 2 < len) { v |= (unsigned)s[i + 2]; n = 3; }
28     if (o + 4 >= out_sz) return -1;
29     out[o++] = T[(v >> 18) & 63];
30     out[o++] = T[(v >> 12) & 63];
31     out[o++] = (n >= 2) ? T[(v >> 6) & 63] : '=';
32     out[o++] = (n >= 3) ? T[v & 63] : '=';
33 }
34 if (o >= out_sz) return -1;
35 out[o] = '\0';
36 return (int)o;
37 }

38 typedef struct {
39     bool https;
40     char host[256];
41     uint16_t port;
42     char path[512];
43 } url_t;
44
45 static inline int parse_url(const char *arg, url_t *u)
46 {
47     char buf[1024];
48     if (strlen(arg) >= sizeof(buf)) return -1;
49     strcpy(buf, arg);
50     char *p = buf;
51     u->https = false;
52     if (strncmp(p, "https://", 8) == 0) { u->https = true; p += 8; }
53     else if (strncmp(p, "http://", 7) == 0) { p += 7; }
54     char *sep = strchr(p, '/');
55     char *hostport = p;
56     if (sep) {
57         if (*sep == '?') {
58             size_t ql = strlen(sep);
59             if (ql + 2 > sizeof(u->path)) return -1;
60             u->path[0] = '/';
61             memcpy(u->path + 1, sep, ql + 1);
62         } else {
63             if (strlen(sep) >= sizeof(u->path)) return -1;
64             strcpy(u->path, sep);
65         }
66         *sep = '\0';
67     } else strcpy(u->path, "/");
68     char *host = u->host;
69     size_t hmax = sizeof(u->host) - 1;
70     uint16_t def = u->https ? 443 : 80;
71     u->port = def;
72     if (*hostport == '[') {
73         char *cl = strchr(hostport, ']');
74         if (!cl) return -1;
75         size_t hl = cl - (hostport + 1);
76         if (hl >= hmax) return -1;
77         memcpy(host, hostport + 1, hl);
78         host[hl] = '\0';
79         if (*(cl + 1) == ':')
80             u->port = (uint16_t)strtol(cl + 2, NULL, 10);
81     } else {
82         char *col = strrchr(hostport, ':');
83         if (col) {
84             size_t hl = col - hostport;
85             if (hl >= hmax) return -1;
86             memcpy(host, hostport, hl);
87             host[hl] = '\0';
88             u->port = (uint16_t)strtol(col + 1, NULL, 10);

```

```

89     } else {
90         if (strlen(hostport) >= hmax) return -1;
91         strcpy(host, hostport);
92     }
93 }
94 if (u->port == 0) u->port = def;
95 return 0;
96 }
97
98 static int socks5_connect(int sock, const char *host, uint16_t port)
99 {
100     unsigned char buf[512];
101     size_t hlen = strlen(host);
102     if (hlen > 255) return -1;
103     buf[0] = 0x05; buf[1] = 0x01; buf[2] = 0x00;
104     if (write(sock, buf, 3) != 3) return -1;
105     if (read(sock, buf, 2) != 2 || buf[0] != 0x05 || buf[1] != 0x00)
106         return -1;
107     buf[0] = 0x05; buf[1] = 0x01; buf[2] = 0x00; buf[3] = 0x03;
108     buf[4] = (unsigned char)hlen;
109     memcpy(buf + 5, host, hlen);
110     buf[5 + hlen] = port >> 8;
111     buf[6 + hlen] = port & 0xff;
112     if (write(sock, buf, 7 + hlen) != (ssize_t)(7 + hlen)) return -1;
113     if (read(sock, buf, 4) != 4 || buf[1] != 0x00) return -1;
114     return 0;
115 }
116
117 static inline bool https_setup(int sock, const char *hostname,
118                               SSL_CTX **out_ctx, SSL **out_ssl)
119 {
120     SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());
121     if (!ctx) { ERR_print_errors_fp(stderr); return false; }
122     SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL);
123     SSL *ssl = SSL_new(ctx);
124     if (!ssl) { SSL_CTX_free(ctx); return false; }
125     SSL_set_fd(ssl, sock);
126     SSL_set_tlsext_host_name(ssl, hostname);
127     if (SSL_connect(ssl) <= 0) {
128         ERR_print_errors_fp(stderr);
129         SSL_free(ssl); SSL_CTX_free(ctx);
130         return false;
131     }
132     *out_ctx = ctx;
133     *out_ssl = ssl;
134     return true;
135 }
136
137 static inline void usage(const char *progname)
138 {
139     fprintf(stderr,
140         "Usage: %s [options] <URL>\n\n"
141         "Options:\n"
142         "  -A, --user-agent STR      Set User-Agent\n"
143         "  -T, --tor [proxy]         Use Tor (default 127.0.0.1:9050)\n"
144         "  -H, --header STR          Add custom header (repeatable)\n"
145         "  -d, --data DATA          POST data (@file supported)\n"
146         "  -o, --output FILE          Write output to file\n"
147         "  -u, --user USER:PASS      Basic authentication\n"
148         "  -I, --head                 HEAD request\n"
149         "      --no-location          Do not follow redirects\n"
150         "      --max-redirects N      Max redirects (default: 10)\n",
151         progname);
152     exit(EXIT_FAILURE);
153 }

```

```

154
155 int main(int argc, char *argv[])
156 {
157     const char *user_agent =
158         "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) "
159         "Gecko/20100101 Firefox/128.0";
160     const char *default_tor = "127.0.0.1:9050";
161     const char *url_arg = NULL;
162     const char *tor_proxy = NULL;
163     bool use_tor = false;
164     bool follow_redirects = true;
165     int max_redirects = 10;
166     bool head_request = false;
167     char extra_headers[4096] = {0};
168     char post_data[8192] = {0};
169     char output_file[256] = {0};
170     char auth_header[256] = {0};
171
172     for (int i = 1; i < argc; i++) {
173         if (strcmp(argv[i], "-A") == 0 ||
174             strcmp(argv[i], "--user-agent") == 0) {
175             if (i + 1 < argc) user_agent = argv[++i];
176             else usage(argv[0]);
177         }
178         else if (strcmp(argv[i], "-T") == 0 ||
179                 strcmp(argv[i], "--tor") == 0) {
180             use_tor = true;
181             if (i + 1 < argc && argv[i + 1][0] != '-' &&
182                 strchr(argv[i + 1], ':')) {
183                 tor_proxy = argv[i + 1];
184                 i++;
185             }
186         }
187         else if (strcmp(argv[i], "-H") == 0 ||
188                 strcmp(argv[i], "--header") == 0) {
189             if (i + 1 < argc) {
190                 strncat(extra_headers, argv[++i],
191                     sizeof(extra_headers) -
192                     strlen(extra_headers) - 3);
193                 strcat(extra_headers, "\r\n");
194             } else usage(argv[0]);
195         }
196         else if (strcmp(argv[i], "-d") == 0 ||
197                 strcmp(argv[i], "--data") == 0) {
198             if (i + 1 < argc) {
199                 const char *val = argv[++i];
200                 if (val[0] == '@') {
201                     FILE *f = fopen(val + 1, "r");
202                     if (f) {
203                         fread(post_data, 1,
204                             sizeof(post_data) - 1, f);
205                         fclose(f);
206                     }
207                 } else
208                     strncpy(post_data, val, sizeof(post_data) - 1);
209             } else usage(argv[0]);
210         }
211         else if (strcmp(argv[i], "-o") == 0 ||
212                 strcmp(argv[i], "--output") == 0) {
213             if (i + 1 < argc)
214                 strncpy(output_file, argv[++i],
215                     sizeof(output_file) - 1);
216             else usage(argv[0]);
217         }

```

```

218     else if (strcmp(argv[i], "-u") == 0 ||
219              strcmp(argv[i], "--user") == 0) {
220         if (i + 1 < argc) {
221             char b64[200];
222             const char *cred = argv[++i];
223             if (base64_encode(cred, strlen(cred), b64,
224                              sizeof(b64)) < 0) {
225                 fprintf(stderr, "credentials too long\n");
226                 return EXIT_FAILURE;
227             }
228             snprintf(auth_header, sizeof(auth_header),
229                      "Authorization: Basic %s\r\n", b64);
230         } else usage(argv[0]);
231     }

232     else if (strcmp(argv[i], "-I") == 0 ||
233              strcmp(argv[i], "--head") == 0)
234         head_request = true;
235     else if (strcmp(argv[i], "--no-location") == 0)
236         follow_redirects = false;
237     else if (strcmp(argv[i], "--max-redirects") == 0 &&
238              i + 1 < argc)
239         max_redirects = atoi(argv[++i]);
240     else if (url_arg == NULL)
241         url_arg = argv[i];
242 }

243
244 if (url_arg == NULL) usage(argv[0]);
245 if (use_tor && tor_proxy == NULL) tor_proxy = default_tor;

246 FILE *out = stdout;
247 if (output_file[0]) {
248     out = fopen(output_file, "w");
249     if (!out) { perror("fopen"); return EXIT_FAILURE; }
250 }

251
252 int redirect_count = 0;
253 char current_url[1024];
254 strncpy(current_url, url_arg, sizeof(current_url) - 1);
255
256 while (redirect_count <= max_redirects) {
257     url_t u = {0};
258     if (parse_url(current_url, &u) != 0) {
259         fprintf(stderr, "Failed to parse URL\n");
260         return EXIT_FAILURE;
261     }
262
263     const char *connect_host = use_tor ? tor_proxy : u.host;
264     uint16_t connect_port = use_tor ? 9050 : u.port;
265     char pbuf[256];
266     if (use_tor) {
267         strncpy(pbuf, tor_proxy, sizeof(pbuf) - 1);
268         char *colon = strrchr(pbuf, ':');
269         if (colon) {
270             *colon = '\0';
271             connect_port =
272                 (uint16_t)strtol(colon + 1, NULL, 10);
273             connect_host = pbuf;
274         }
275     }

276
277     struct addrinfo hints = {0};
278     hints.ai_family = AF_UNSPEC;
279     hints.ai_socktype = SOCK_STREAM;
280     char ps[6];
281     snprintf(ps, sizeof(ps), "%u", connect_port);

```

```

282     struct addrinfo *res = NULL;
283     if (getaddrinfo(connect_host, ps, &hints, &res) != 0) {
284         perror("getaddrinfo");
285         return EXIT_FAILURE;
286     }
287
288
289     int s = -1;
290     for (struct addrinfo *a = res; a; a = a->ai_next) {
291         s = socket(a->ai_family, a->ai_socktype, a->ai_protocol);
292         if (s < 0) continue;
293         if (connect(s, a->ai_addr, a->ai_addrlen) == 0) break;
294         close(s); s = -1;
295     }
296     freeaddrinfo(res);
297     if (s < 0) { perror("connect"); return EXIT_FAILURE; }
298
299     if (use_tor && socks5_connect(s, u.host, u.port) != 0) {
300         close(s);
301         return EXIT_FAILURE;
302     }
303
304     SSL_CTX *ctx = NULL;
305     SSL *ssl = NULL;
306     if (u.https && !https_setup(s, u.host, &ctx, &ssl)) {
307         close(s);
308         return EXIT_FAILURE;
309     }
310
311     const char *method = head_request ? "HEAD"
312                                     : (post_data[0] ? "POST" : "GET");
313
314     char req[4096];
315     int len = snprintf(req, sizeof(req),
316                       "%s %s HTTP/1.1\r\n"
317                       "Host: %s\r\n"
318                       "User-Agent: %s\r\n"
319                       "%s"
320                       "%s"
321                       "Accept: */*\r\n"
322                       "Connection: close\r\n",
323                       method, u.path, u.host, user_agent,
324                       extra_headers, auth_header);
325
326     if (post_data[0]) {
327         len += snprintf(req + len, sizeof(req) - len,
328                         "Content-Length: %zu\r\n\r\n%s",
329                         strlen(post_data), post_data);
330     } else {
331         strcat(req + len, "\r\n");
332     }
333
334     if (u.https)
335         SSL_write(ssl, req, strlen(req));
336     else
337         write(s, req, strlen(req));
338
339     char buf[8192];
340     ssize_t n;
341     while ((n = u.https
342             ? SSL_read(ssl, buf, sizeof(buf))
343             : read(s, buf, sizeof(buf))) > 0)
344         fwrite(buf, 1, n, out);
345
346     if (u.https) {

```

```
347         if (ssl) { SSL_shutdown(ssl); SSL_free(ssl); }
348         if (ctx) SSL_CTX_free(ctx);
349     }
350     close(s);
351
352     if (follow_redirects && strstr(buf, "Location:") &&
353         redirect_count < max_redirects) {
354         char location[1024] = {0};
355         char *loc = strstr(buf, "Location:");
356         if (loc) sscanf(loc, "Location: %1023s", location);
357         if (location[0]) {
358             redirect_count++;
359             strncpy(current_url, location,
360                     sizeof(current_url) - 1);
361             continue;
362         }
363     }
364     break;
365 }
366
367 if (out != stdout) fclose(out);
368
369 return 0;
370 }
```

```
1 ./urlget -I https://example.com/
2 ./urlget -H "Accept-Language: en" https://example.com/
3 ./urlget -d "name=Ada" -o resp.txt https://httpbin.org/post
4 # curl-style USER:PASS --- encoded inside urlget
5 ./urlget -u 'user:pass' https://httpbin.org/basic-auth/user/pass
```

Step 10: Final Polish and Release Build

The functional client is complete. This step adds structured debug tracing (compile-time removable), feature-test macros for POSIX APIs, and presents the full production-shaped `urlget.c`.

Debug Infrastructure

The real program includes a small header `debug.h` controlled by:

- `DEBUG_LEVEL` — verbosity threshold
- `DEBUG_COLORS` / `DEBUG_TIMESTAMPS` — presentation
- `-DNDEBUG` — strips all debug calls for release builds

Typical macros used in the final source:

```
1 DEBUG_ENTER();
2 DEBUG_ENTER("(arg=%s)", arg);
3 DEBUG_TRACE("Connected to %s:%u", host, port);
4 DEBUG_ERROR("parse_url failed");
```

If you do not have `debug.h`, a minimal stub is enough to compile:

```
1 /* debug.h -- minimal stub */
2 #ifndef DEBUG_H
3 #define DEBUG_H
4 #ifdef NDEBUG
5 # define DEBUG_ENTER(...) ((void)0)
6 # define DEBUG_TRACE(...) ((void)0)
7 # define DEBUG_ERROR(...) ((void)0)
8 #else
9 # include <stdio.h>
10 # define DEBUG_ENTER(...) fprintf(stderr, "ENTER %s\n", __func__)
11 # define DEBUG_TRACE(...) do { fprintf(stderr, "TRACE: "); \
12     fprintf(stderr, __VA_ARGS__); fprintf(stderr, "\n"); } while (0)
13 # define DEBUG_ERROR(...) do { fprintf(stderr, "ERROR: "); \
14     fprintf(stderr, __VA_ARGS__); fprintf(stderr, "\n"); } while (0)
15 #endif
16 #endif
```

Compile Commands

```
1 # Development (debug traces on)
2 gcc -Wall -Wextra -O0 -g -o urlget urlget.c -lssl -lcrypto
3
4 # Release (no debug noise)
5 gcc -Wall -Wextra -O2 -DNDEBUG -o urlget urlget.c -lssl -lcrypto
```

Feature Map (where everything lives)

Concern	Location in final source
URL → host/port/path	url_t, parse_url
CLI options	usage, main arg loop
Plain TCP	getaddrinfo / socket / connect
TLS	https_setup, SSL_read/write
Tor	socks5_connect, -T path
Redirects	outer while + Location: scan
POST / HEAD / headers / -o	request builder + out FILE
Basic auth (-u)	base64_encode + Authorization header

Final Complete Source

The listing below is the complete `urlget.c` as developed in this tutorial (debug calls included). Study it as one coherent program—every piece was introduced with a reason in an earlier step.

```

1  #define _POSIX_C_SOURCE 200809L
2  #define _GNU_SOURCE
3
4  #define DEBUG_LEVEL 5
5  #define DEBUG_COLORS
6  #define DEBUG_TIMESTAMPS
7  #include "debug.h"
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <string.h>
12 #include <stdbool.h>
13 #include <stdint.h>
14 #include <unistd.h>
15 #include <sys/types.h>
16 #include <sys/socket.h>
17 #include <netdb.h>
18 #include <openssl/ssl.h>
19 #include <openssl/err.h>
20 #include <arpa/inet.h>
21
22 typedef struct {
23     bool https;
24     char host[256];
25     uint16_t port;
26     char path[512];
27 } url_t;
28
29 static inline int parse_url(const char *arg, url_t *u)
30 {
31     DEBUG_ENTER("(arg=%s)", arg);
32     char buf[1024];
33     if (strlen(arg) >= sizeof(buf)) {
34         DEBUG_ERROR("URL too long");
35         return -1;
36     }
37     strcpy(buf, arg);
38
39     char *p = buf;
40     u->https = false;
41
42     if (strncmp(p, "https://", 8) == 0) { u->https = true; p += 8; }
43     else if (strncmp(p, "http://", 7) == 0) { p += 7; }
44
45     char *sep = strpbrk(p, "/?");

```

```

46     char *hostport = p;
47
48     if (sep) {
49         if (*sep == '?') {
50             size_t ql = strlen(sep);
51             if (ql + 2 > sizeof(u->path)) return -1;
52             u->path[0] = '/';
53             memcpy(u->path + 1, sep, ql + 1);
54         } else {
55             if (strlen(sep) >= sizeof(u->path)) return -1;
56             strcpy(u->path, sep);
57         }
58         *sep = '\0';
59     } else {
60         strcpy(u->path, "/");
61     }
62
63     char *host = u->host;
64     size_t hmax = sizeof(u->host) - 1;
65     uint16_t def = u->https ? 443 : 80;
66     u->port = def;
67
68     if (*hostport == '[') {
69         char *cl = strchr(hostport, ']');
70         if (!cl) return -1;
71         size_t hl = cl - (hostport + 1);
72         if (hl >= hmax) return -1;
73         memcpy(host, hostport + 1, hl);
74         host[hl] = '\0';
75         char *aft = cl + 1;
76         if (*aft == ':')
77             u->port = (uint16_t)strtol(aft + 1, NULL, 10);
78     } else {
79         char *col = strrchr(hostport, ':');
80         if (col) {
81             size_t hl = col - hostport;
82             if (hl >= hmax) return -1;
83             memcpy(host, hostport, hl);
84             host[hl] = '\0';
85             u->port = (uint16_t)strtol(col + 1, NULL, 10);
86         } else {
87             if (strlen(hostport) >= hmax) return -1;
88             strcpy(host, hostport);
89         }
90     }
91
92     if (u->port == 0) u->port = def;
93
94     DEBUG_TRACE("Parsed: https=%d host=%s port=%u path=%s",
95                 u->https, u->host, u->port, u->path);
96     return 0;
97 }
98
99 static int socks5_connect(int sock, const char *host, uint16_t port)
100 {
101     unsigned char buf[512];
102     size_t hlen = strlen(host);
103     if (hlen > 255) {
104         DEBUG_ERROR("Hostname too long");
105         return -1;
106     }
107
108     DEBUG_TRACE("SOCKS5 -> Greeting");
109     buf[0] = 0x05; buf[1] = 0x01; buf[2] = 0x00;
110     if (write(sock, buf, 3) != 3) return -1;

```

```

111
112     if (read(sock, buf, 2) != 2 || buf[0] != 0x05 || buf[1] != 0x00) {
113         DEBUG_ERROR("SOCKS5 greeting rejected");
114         return -1;
115     }
116
117     DEBUG_TRACE("SOCKS5 -> Connect request to %s:%u", host, port);
118     buf[0] = 0x05; buf[1] = 0x01; buf[2] = 0x00; buf[3] = 0x03;
119     buf[4] = (unsigned char)hlen;
120     memcpy(buf + 5, host, hlen);
121     buf[5 + hlen] = port >> 8;
122     buf[6 + hlen] = port & 0xff;
123
124     if (write(sock, buf, 7 + hlen) != (ssize_t)(7 + hlen)) return -1;
125
126     if (read(sock, buf, 4) != 4 || buf[1] != 0x00) {
127         DEBUG_ERROR("SOCKS5 connect failed (REP=0x%02x)", buf[1]);
128         return -1;
129     }
130
131     DEBUG_TRACE("SOCKS5 handshake successful");
132     return 0;
133 }
134
135 static inline bool https_setup(int sock, const char *hostname,
136                               SSL_CTX **out_ctx, SSL **out_ssl)
137 {
138     SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());
139     if (!ctx) { ERR_print_errors_fp(stderr); return false; }
140
141     SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL);
142
143     SSL *ssl = SSL_new(ctx);
144     if (!ssl) { SSL_CTX_free(ctx); return false; }
145
146     SSL_set_fd(ssl, sock);
147     SSL_set_tlsext_host_name(ssl, hostname);
148
149     if (SSL_connect(ssl) <= 0) {
150         ERR_print_errors_fp(stderr);
151         SSL_free(ssl); SSL_CTX_free(ctx);
152         return false;
153     }
154
155     *out_ctx = ctx;
156     *out_ssl = ssl;
157     return true;
158 }
159
160 /* RFC 4648 Base64 encoder for HTTP Basic auth (-u USER:PASS). */
161 static inline int base64_encode(const void *data, size_t len,
162                                char *out, size_t out_sz)
163 {
164     static const char T[] =
165         "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"
166         "0123456789+/";
167     const unsigned char *s = data;
168     size_t o = 0;
169
170     for (size_t i = 0; i < len; i += 3) {
171         unsigned v = (unsigned)s[i] << 16;
172         int n = 1;
173         if (i + 1 < len) { v |= (unsigned)s[i + 1] << 8; n = 2; }
174         if (i + 2 < len) { v |= (unsigned)s[i + 2]; n = 3; }
175         if (o + 4 >= out_sz) return -1;

```

```

176     out[o++] = T[(v >> 18) & 63];
177     out[o++] = T[(v >> 12) & 63];
178     out[o++] = (n >= 2) ? T[(v >> 6) & 63] : '=';
179     out[o++] = (n >= 3) ? T[v & 63] : '=';
180 }
181 if (o >= out_sz) return -1;
182 out[o] = '\0';
183 return (int)o;
184 }
185
186 static inline void usage(const char *programe)
187 {
188     fprintf(stderr,
189         "Usage: %s [options] <URL>\n\n"
190         "Options:\n"
191         "  -A, --user-agent STR      Set User-Agent\n"
192         "  -T, --tor [proxy]         Use Tor (default 127.0.0.1:9050)\n"
193         "  -H, --header STR          Add custom header (repeatable)\n"
194         "  -d, --data DATA          POST data (@file supported)\n"
195         "  -o, --output FILE         Write output to file\n"
196         "  -u, --user USER:PASS     Basic authentication\n"
197         "  -I, --head                HEAD request\n"
198         "      --no-location         Do not follow redirects\n"
199         "      --max-redirects N     Max redirects (default: 10)\n",
200         programe);
201     exit(EXIT_FAILURE);
202 }
203
204 int main(int argc, char *argv[])
205 {
206     DEBUG_ENTER();
207
208     const char *user_agent =
209         "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) "
210         "Gecko/20100101 Firefox/128.0";
211     const char *default_tor = "127.0.0.1:9050";
212
213     const char *url_arg = NULL;
214     const char *tor_proxy = NULL;
215     bool use_tor = false;
216     bool follow_redirects = true;
217     int max_redirects = 10;
218     bool head_request = false;
219     char extra_headers[4096] = {0};
220     char post_data[8192] = {0};
221     char output_file[256] = {0};
222     char auth_header[256] = {0};
223
224     for (int i = 1; i < argc; i++) {
225         if (strcmp(argv[i], "-A") == 0 ||
226             strcmp(argv[i], "--user-agent") == 0) {
227             if (i + 1 < argc) user_agent = argv[i + 1];
228             else usage(argv[0]);
229         }
230         else if (strcmp(argv[i], "-T") == 0 ||
231             strcmp(argv[i], "--tor") == 0) {
232             use_tor = true;
233             if (i + 1 < argc && argv[i + 1][0] != '-' &&
234                 strchr(argv[i + 1], ':')) {
235                 tor_proxy = argv[i + 1];
236                 i++;
237             }
238         }
239         else if (strcmp(argv[i], "-H") == 0 ||
240             strcmp(argv[i], "--header") == 0) {

```

```

241     if (i + 1 < argc) {
242         strncat(extra_headers, argv[++i],
243             sizeof(extra_headers) -
244             strlen(extra_headers) - 3);
245         strcat(extra_headers, "\r\n");
246     } else usage(argv[0]);
247 }
248 else if (strcmp(argv[i], "-d") == 0 ||
249     strcmp(argv[i], "--data") == 0) {
250     if (i + 1 < argc) {
251         const char *val = argv[++i];
252         if (val[0] == '@') {
253             FILE *f = fopen(val + 1, "r");
254             if (f) {
255                 fread(post_data, 1,
256                     sizeof(post_data) - 1, f);
257                 fclose(f);
258             }
259         } else {
260             strncpy(post_data, val, sizeof(post_data) - 1);
261         }
262     } else usage(argv[0]);
263 }
264 else if (strcmp(argv[i], "-o") == 0 ||
265     strcmp(argv[i], "--output") == 0) {
266     if (i + 1 < argc)
267         strncpy(output_file, argv[++i],
268             sizeof(output_file) - 1);
269     else usage(argv[0]);
270 }
271 else if (strcmp(argv[i], "-u") == 0 ||
272     strcmp(argv[i], "--user") == 0) {
273     if (i + 1 < argc) {
274         char b64[200];
275         const char *cred = argv[++i];
276         if (base64_encode(cred, strlen(cred), b64,
277             sizeof(b64)) < 0) {
278             fprintf(stderr, "credentials too long\n");
279             return EXIT_FAILURE;
280         }
281         snprintf(auth_header, sizeof(auth_header),
282             "Authorization: Basic %s\r\n", b64);
283     } else usage(argv[0]);
284 }
285 else if (strcmp(argv[i], "-I") == 0 ||
286     strcmp(argv[i], "--head") == 0) {
287     head_request = true;
288 }
289 else if (strcmp(argv[i], "--no-location") == 0) {
290     follow_redirects = false;
291 }
292 else if (strcmp(argv[i], "--max-redirects") == 0 &&
293     i + 1 < argc) {
294     max_redirects = atoi(argv[++i]);
295 }
296 else if (url_arg == NULL) {
297     url_arg = argv[i];
298 }
299 }
300
301 if (url_arg == NULL) usage(argv[0]);
302 if (use_tor && tor_proxy == NULL) tor_proxy = default_tor;
303
304 FILE *out = stdout;
305 if (output_file[0]) {

```

```

306     out = fopen(output_file, "w");
307     if (!out) { perror("fopen"); return EXIT_FAILURE; }
308 }
309
310 int redirect_count = 0;
311 char current_url[1024];
312 strncpy(current_url, url_arg, sizeof(current_url) - 1);
313
314 while (redirect_count <= max_redirects) {
315     url_t u = {0};
316     if (parse_url(current_url, &u) != 0) {
317         DEBUG_ERROR("parse_url failed");
318         return EXIT_FAILURE;
319     }
320
321     DEBUG_TRACE("Using Tor: %s | Redirects: %d/%d",
322                use_tor ? "yes" : "no",
323                redirect_count, max_redirects);
324
325     const char *connect_host = use_tor ? tor_proxy : u.host;
326     uint16_t connect_port = use_tor ? 9050 : u.port;
327
328     if (use_tor) {
329         char pbuf[256];
330         strncpy(pbuf, tor_proxy, sizeof(pbuf) - 1);
331         char *colon = strchr(pbuf, ':');
332         if (colon) {
333             *colon = '\0';
334             connect_port =
335                 (uint16_t)strtol(colon + 1, NULL, 10);
336             connect_host = pbuf;
337         }
338     }
339
340     struct addrinfo hints = {0};
341     hints.ai_family = AF_UNSPEC;
342     hints.ai_socktype = SOCK_STREAM;
343     char ps[6];
344     snprintf(ps, sizeof(ps), "%u", connect_port);
345
346     struct addrinfo *res = NULL;
347     if (getaddrinfo(connect_host, ps, &hints, &res) != 0) {
348         perror("getaddrinfo");
349         return EXIT_FAILURE;
350     }
351
352     int s = -1;
353     for (struct addrinfo *a = res; a; a = a->ai_next) {
354         s = socket(a->ai_family, a->ai_socktype, a->ai_protocol);
355         if (s < 0) continue;
356         if (connect(s, a->ai_addr, a->ai_addrlen) == 0) break;
357         close(s); s = -1;
358     }
359     freeaddrinfo(res);
360     if (s < 0) { perror("connect"); return EXIT_FAILURE; }
361
362     DEBUG_TRACE("Connected to %s:%u%s", connect_host, connect_port,
363                use_tor ? " (Tor)" : "");
364
365     if (use_tor && socks5_connect(s, u.host, u.port) != 0) {
366         close(s);
367         return EXIT_FAILURE;
368     }
369
370     SSL_CTX *ctx = NULL;

```

```

371     SSL *ssl = NULL;
372
373     if (u.https && !https_setup(s, u.host, &ctx, &ssl)) {
374         close(s);
375         return EXIT_FAILURE;
376     }
377
378     const char *method = head_request ? "HEAD"
379                                     : (post_data[0] ? "POST" : "GET");
380
381     char req[4096];
382     int len = snprintf(req, sizeof(req),
383                       "%s %s HTTP/1.1\r\n"
384                       "Host: %s\r\n"
385                       "User-Agent: %s\r\n"
386                       "%s"
387                       "%s"
388                       "Accept: */*\r\n"
389                       "Connection: close\r\n",
390                       method, u.path, u.host, user_agent,
391                       extra_headers, auth_header);
392
393     if (post_data[0]) {
394         len += snprintf(req + len, sizeof(req) - len,
395                       "Content-Length: %zu\r\n\r\n%s",
396                       strlen(post_data), post_data);
397     } else {
398         strcat(req + len, "\r\n");
399     }
400
401     if (u.https)
402         SSL_write(ssl, req, strlen(req));
403     else
404         write(s, req, strlen(req));
405
406     DEBUG_TRACE("Request sent (%s)", method);
407
408     char buf[8192];
409     ssize_t n;
410     while ((n = u.https
411             ? SSL_read(ssl, buf, sizeof(buf))
412             : read(s, buf, sizeof(buf))) > 0) {
413         fwrite(buf, 1, n, out);
414     }
415
416     if (u.https) {
417         if (ssl) { SSL_shutdown(ssl); SSL_free(ssl); }
418         if (ctx) SSL_CTX_free(ctx);
419     }
420     close(s);
421
422     if (follow_redirects && strstr(buf, "Location:") &&
423         redirect_count < max_redirects) {
424         char location[1024] = {0};
425         char *loc = strstr(buf, "Location:");
426         if (loc) sscanf(loc, "Location: %1023s", location);
427
428         if (location[0]) {
429             redirect_count++;
430             DEBUG_TRACE("Following redirect %d/%d -> %s",
431                         redirect_count, max_redirects, location);
432             strncpy(current_url, location,
433                     sizeof(current_url) - 1);
434             continue;
435         }

```

```
436     }
437
438     break;
439 }
440
441 if (out != stdout) fclose(out);
442 DEBUG_TRACE("Done");
443 return 0;
444 }
```

Closing Thoughts

You now have a single-file HTTP(S) client that:

- Parses URLs and options explicitly
- Speaks TCP, optional TLS, and optional SOCKS5
- Follows redirects with a safety limit
- Supports the common request customizations of a mini-curl, including curl-style `-u USER:PASS` with built-in Base64

By cloc, the finished `urlget.c` is about **348 lines of code** (plus a handful of blanks and comments). Not bad for a pocket-sized mini-curl.

Natural next exercises: verify TLS certificates properly, parse HTTP status lines and headers fully, and support chunked transfer encoding.